
Titulo del TFG
Title in english

Trabajo de Fin de Grado
Curso 2025–2026

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

Titulo del TFG
Title in english

Trabajo de Fin de Grado en Desarrollo de Videojuegos

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Convocatoria: *Junio 2026*

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

19 de mayo de 2026

Dedicatoria

*A toda la gente que nos ha hecho llegar hasta
aquí,
un besazo.
A todos los que han estado a nuestro lado.*

Agradecimientos

Os amo!

Mik

Os amo!

Pau

Os amo!

Eva

Resumen

Titulo del TFG

Nuestro estudio se basa en la comparativa de distintos modelos de IA para hacer frente a la simulación de físicas costosas en proyectos 3D en motores de videojuegos.

Para ello creamos una simulación en Unity3D con un objeto con componente Cloth y una esfera que actúe como colisionador con la tela. Tras generar los datasets, los exportamos a cuadernos de Jupyter dónde los entrenamos con diferentes modelos en Pytorch (con soporte GPU): una red neuronal sencilla, una RNN, un Encoder-Decoder.

Finalmente exportamos las redes entrenadas a Unity mediante el paquete de Sentis para simular el movimiento modificando los vértices de una tela sin componente Cloth. Realizamos comparativas entre los distintos modelos, tanto en cuanto a precisión del modelo como a resultados visuales.

Observamos que el modelo que mejores resultados obtuvo en cuanto a tal métrica fue X.

Palabras clave

Inteligencia Artificial, Simulación física, Unity, Videojuegos, Optimización, PyTorch, ML

Abstract

Title in english

We need to do this.

Keywords

We need to do this.

Índice

1. Introducción	1
2. Estado de la Cuestión	3
2.1. Motores de videojuegos	3
2.2. Motores de física	4
2.3. Simulación física en videojuegos	5
2.4. Simulación de telas	6
2.4.1. Cloth	6
2.4.2. Optimizaciones en simulación de telas	7
2.5. IA para optimización de telas	8
2.5.1. Modelos de optimización de telas	8
2.5.1.1. Subspace Neural Physics: Fast Data-Driven Interac- tive Simulation	9
2.5.1.2. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network	9
2.5.1.3. PBNS: Physically Based Neural Simulation for Un- supervised Garment Pose Space Deformation	10
2.5.1.4. SNUG	11
2.5.1.5. Neural Cloth Simulation	11
2.5.1.6. HOOD: Hierarchical Graphs for Generalized Mode- ling of Clothing Dynamics	11
2.5.2. Librerías	12
2.5.2.1. Pytorch	12
2.5.2.2. ONNX	12
2.5.2.3. Sentis	12
3. Descripción del Trabajo	15
3.1. Arquitectura General	15
3.2. El dataset	16
3.2.1. La simulación	16
3.2.2. Otras simulaciones	17
3.2.3. Los parámetros	20

3.2.4.	Generación del dataset	21
3.3.	Entrenamiento	21
3.3.1.	El entorno de trabajo	21
3.3.2.	Apartados de la ejecución del entrenamiento	22
3.3.3.	Los modelos	24
3.3.3.1.	Modelo inicial	24
3.3.3.2.	Adaptaciones del modelo	25
3.3.4.	Parametrización	25
3.3.5.	Gestión del error acumulado	27
3.3.6.	Resultado e hiperparámetros	29
3.4.	SimulacionUnity	32
3.4.1.	Alteraciones en la escena	32
3.4.2.	Componente ClothML	32
3.4.2.1.	Funcionamiento de sentis	32
3.4.2.2.	Estructura del componente	33
3.4.3.	Resultados visuales de la inferencia	35
3.4.4.	Post-procesamiento	35
4.	Conclusiones y Trabajo Futuro	37
4.1.	Conclusiones	37
4.1.1.	Limitaciones	38
4.2.	Trabajo Futuro	39
	Contribuciones Personales	41
	Bibliografía	43
	Licencia de uso del documento	47
	Licencia de uso del código fuente	49

Índice de figuras

3.1. Diagrama de la arquitectura General del proyecto	15
3.2. Malla de 25 vértices con parte superior fija.	17
3.3. Movimiento aleatorio basado en splines.	18
3.4. Simulación de una camiseta con componente Cloth.	19
3.5. Simulación de una falda con componente Cloth.	19
3.6. Simulación de una falda con componente Cloth.	19
3.7. Visualización de los puntos de la tela simulada e inferida a lo largo del tiempo	23
3.8. Visualización del error medio a lo largo del tiempo	24
3.9. Visualización de error de normalización	24
3.10. Parámetros valorados por el random forest	26
3.11. Reducción de características	27
3.12. Reducción de características	27
3.13. Esquema del contenido de FixedUpdate() de ClothML.	34

Índice de tablas

3.1. Librerías y versiones del entorno de desarrollo.	22
---	----

Introducción

En los últimos años, el aumento en el tamaño y la complejidad en los videojuegos ha supuesto la imprescindible optimización y automatización de diversos aspectos de su desarrollo, tanto en el ámbito del renderizado como en la lógica interna de los sistemas. Particularmente la evolución de entornos 3D, con el objetivo de obtener gráficos más detallados, ha implicado un incremento significativo en el coste computacional. Una forma de afrontar estas simulaciones para reducir su impacto computacional, es utilizar modelos de ML que permiten aproximar el comportamiento de estos elementos reduciendo el coste.

Empresas pioneras en hardware y software han desarrollado tecnologías para mejorar el rendimiento gráfico incorporando esta clase de algoritmos. Entre ellas destacan: NVIDIA con DLSS (Deep Learning Super Sampling), donde tenemos el Resident Evil Requiem que usa DLSS 4¹; y AMD con FSR (FidelityFX Super Resolution)². Asimismo, varias compañías lo incorporan en la lógica del juego simulando comportamientos y mecánicas como EA (FIFA), Embark Studios (Arc Raiders) o Ubisoft con sus múltiples líneas de investigación sobre la IA. En este contexto, un tercer campo de aplicación que está en proceso de investigación es la aplicación de modelos de aprendizaje automático en simulaciones físicas.

En el campo de la simulación física, con el incremento de la capacidad de cálculo de los sistemas actuales, se busca una mayor fidelidad en la recreación de las interacciones entre los diferentes elementos. Así por ejemplo, elementos como el cabello, los fluidos o las telas, elementos que hace poco más de una década estaban muy simplificados, en la actualidad se busca cada vez más con comportamiento realista en sus interacciones. Esta simulación física de sus comportamientos resulta en una carga computacional significativa, lo que afecta negativamente al rendimiento de

¹**Deep Learning Super Sampling:** Aumenta el rendimiento del videojuego permitiéndolo renderizarse con una resolución más baja, ya que esta tecnología consigue reconstruir la imagen a mayor calidad en runtime. Utiliza herramientas como *Ray Reconstruction* para reemplazar sus *denoisers* manuales o DLAA (Deep Learning Anti-Aliasing) para suavizar los bordes.

²**FidelityFX Super Resolution:** técnica desarrollada por AMD de código abierto que consiste en el aumento del rendimiento en videojuegos mediante el escalado de resolución. Se diferencia del DLSS de NVIDIA por no tener requerimientos específicos de hardware.

los videojuegos, en especial en términos de tasa de fotogramas. La industria se encuentra en un momento de auge en investigación y aplicación de Machine Learning para solucionarlo. Se puede observar a través de los estudios de Ubisoft la Forge, el grupo de desarrollo e investigación de Ubisoft³, donde se aplica ML en distintos campos como la compresión de texturas, comportamiento de NPCs, procesamiento de Lenguaje Natural y Audio o animación y simulación como la simulación de telas Holden et al. (2019).

1.1. Motivación

En la misma línea de otras tecnologías ya utilizadas en el desarrollo de videojuegos como la mencionada DLSS de NVIDIA, donde se aplican modelos de redes neuronales profundas para reducir la carga computacional de los juegos en el renderizado, se conducen estudios y empiezan a aplicar técnicas similares en la industria en simulación física. Nace en un contexto donde los juegos tienden a ser distribuidos ya no solo en consolas con hardware y gráficas dedicadas, sino en múltiples plataformas con el objeto de maximizar ingresos, donde no todas tienen las mismas capacidades de cómputo. El uso de estas técnicas de optimización puede permitir recrear estas simulaciones con un grado de fidelidad aceptable, en dispositivos de menor potencia como pueden ser Nintendo Switch, Handle pcs o dispositivos móviles.

Este enfoque implica una carga elevada al inicio de entrenamiento de modelos, compensado con una ejecución eficiente en tiempo real. Cabe destacar que, si bien utilizar la IA supone sacrificar la precisión absoluta de las físicas, no queremos caer en la falacia de la inmersión Salen y Zimmerman (2004). Nuestro objetivo es lograr simulaciones visualmente verosímiles, centrándonos en la percepción del usuario. Por ello, se ha optado por la simulación de telas. Este caso de estudio representa un equilibrio adecuado entre complejidad física y familiaridad con el resultado visual, de esta manera, realizaremos un análisis tanto técnico como perceptivo.

1.2. Objetivos

El objetivo general de este proyecto es la implementación de un modelo de IA que permita replicar el movimiento en tiempo real de una tela en un motor de videojuegos con baja latencia. Para cumplir con este objetivo, se plantean las siguientes metas durante el desarrollo del proyecto:

1. Generar una animación de una tela y un colisionador que varíe su movimiento en cada ejecución de la escena.
2. Generar y procesar datasets con la información relevante de la tela respecto a su entorno y colisionador.

³Artículos y publicaciones de Ubisoft la Forge: <https://www.ubisoft.com/en-us/studio/laforge/publications#24A8zwsBgQ1tTM1NRp20b9%C3%A7>

3. Implementar, entrenar y evaluar los distintos modelos de aprendizaje automático, tales como redes neuronales.
4. Analizar y comparar los resultados visuales de los distintos modelos.

1.3. Plan de trabajo

El desarrollo del proyecto se llevará a cabo en el periodo lectivo del curso 2025-2026. Dados los objetivos del apartado anterior, la planificación se divide en las siguientes iteraciones:

- **Iteración 0:** En esta etapa se concretan los objetivos reales de la investigación y se establece la planificación del trabajo respecto a la propuesta inicial.
- **Iteración 1:** Etapa de preparación, donde se cumplirán los siguientes subobjetivos para implementar un sistema de simulación adecuado para ser procesado por los modelos en un futuro.
 - **Iteración 1.1:** Se realizará una investigación de trabajos relacionados a este, relevante para dirigir el desarrollo de los experimentos. Se analizarán distintas alternativas y se decidirá cuales resultarán más interesantes de replicar, se definirán qué tipo de datos deberemos recopilar para el entrenamiento y qué tipo de redes podrán llevarlo a cabo.
 - **Iteración 1.2:** Se creará una simulación básica en Unity y un sistema de registro de datos relevantes a la tela.
- **Iteración 2:** Se establecerá un entorno de trabajo para el entrenamiento de los datos recopilados. Se generará un pipeline claro en el que se integre la simulación de recogida de datos, su limpieza y posterior uso para entrenar el modelo y, finalmente, la incorporación de este al entorno de simulación. Para poder asegurar del correcto funcionamiento de este sistema se generará un modelo simple y se comprobará que el pipeline es correcto. En esta fase se comenzarán a redactar los primeros capítulos de la memoria.
- **Iteración 3:** Con una base sólida ya montada, se aumentará la complejidad y, por consiguiente, el coste de la simulación. Se implementarán distintos modelos y se llevarán a cabo pruebas con todos ellos para asegurarse de su fiabilidad.
- **Iteración 4:** Si las anteriores iteraciones funcionan de manera correcta esta última no tendrá ninguna carga de implementación. Se alcanzará la última meta definida: hacer distintas pruebas entrenando los modelos ya desarrollados, sacar métricas de todos ellos para poder hacer comparaciones y hacer un análisis exhaustivo de estos resultados, sacando así las conclusiones de la investigación.

El desarrollo de las tareas y su planificación se puede ver en la figura ??.



Figura 1.1: Diagrama de Gantt del proyecto. Representa la organización de las tareas en el tiempo de desarrollo del proyecto.

1.4. Metodología

Para el desarrollo de esta investigación se emplearán herramientas de libre acceso y uso gratuito, con el objetivo de facilitar la reproducibilidad de los resultados obtenidos. El entorno de desarrollo principal será Windows 11, al tratarse del sistema operativo utilizado por el equipo de trabajo y extendido entre los potenciales usuarios. La simulación de la tela, así como la integración del modelo de inteligencia artificial, se llevarán a cabo en Unity (versión 6000.3.2f1, correspondiente a una versión de soporte a largo plazo). En cuanto a los lenguajes de programación, se empleará C# para el desarrollo dentro de Unity, incluyendo la generación de la simulación y la recopilación de datos, mientras que se utilizará Python con Pytorch para el entrenamiento de los modelos de aprendizaje automático. La integración de los modelos entrenados en el entorno de ejecución se realizará a través del paquete Unity Sentis.

Para la gestión del código fuente y el trabajo colaborativo se utilizará GitHub, basado en el sistema de control de versiones Git. Asimismo, se empleará la herramienta GitHub Projects para la planificación, asignación y seguimiento de tareas. Como herramientas adicionales, se utilizará Google Drive para el almacenamiento de documentación, apuntes de reuniones y materiales de trabajo puntuales, y Discord para la coordinación del equipo mediante reuniones online, además de encuentros presenciales semanales.

Capítulo 2

Estado de la Cuestión

En este capítulo se presenta una breve resumen del desarrollo de las simulaciones físicas en videojuegos, así como el hardware y software que ha hecho que esta evolución sea posible. También se presenta la simulación de telas, sus diferencias de implementación, los problemas de optimización que estas suponen y los avances en modelos de IA que permiten imaginar otras soluciones.

2.1. Motores de videojuegos

Si bien este proyecto se centra en la computación física, gestionada por los motores físicos, los motores de videojuegos proporcionan un conjunto más amplio de herramientas comunes para el desarrollo de videojuegos, entre los cuales encontramos también los motores de renderizado, sonido y otros componentes como los sistemas de scripting o interfaz de usuario, que se complementan para satisfacer las necesidades que requiere la creación de un juego. Existe gran cantidad de motores de videojuegos, siendo algunos de los más conocidos Unity y Unreal, que se detallarán a continuación. Además de estos motores comerciales de acceso general, muchos estudios grandes disponen de motores propios adaptados a sus necesidades y especialización. Algunos ejemplos son Frostbite de Electronic Arts y SnowDrop de Ubisoft. Los dos motores comerciales más utilizados son:

- **Unreal Engine:** Es un motor de videojuegos creado por Epic Games en 1998. Destaca por su motor gráfico, que permite gráficos realistas en tiempo real con tecnologías como Nanite para geometría virtualizada o Lumen para iluminación dinámica. Es multiplataforma y se encuentra no sólo en pequeños desarrollos independientes (indie), sino en grandes producciones multimillonarias (producciones AAA'). Se caracteriza por su sistema de programación visual (Blueprints) en programación con C++. Integraba Physx como motor físico hasta la introducción reciente de Chaos Physics en su versión 5.7, que aporta novedades que se destacarán más adelante en el apartado 2.2.3.
- **Unity:** Creado por Unity Technologies en 2005, abarca un porcentaje del 51 %

de juegos publicados en 2024 y un 26 % ¹ en las unidades vendidas en Steam. Es el motor de videojuegos más utilizado por desarrolladores indie y cuenta con documentación extensa, una gran comunidad y extensiones como Sentis para la integración de modelos de IA, que explicaremos más adelante. Unity3D integra PhysX como motor de físicas, pero puede incorporar otros sistemas de físicas como Havok bajo licencia ². Está basado en la arquitectura Entidad-Componente en C#, y típicamente se utiliza para juegos más pequeños y menos potentes en cuanto a gráficos.

2.2. Motores de física

Un motor de físicas es un conjunto de funciones y recursos que permite la implementación de sistemas de físicas como cuerpos rígidos con su consecuente detección de colisiones, cuerpos blandos e incluso fluidos.

Los motores de física se extendieron en los años 90, con la popularidad del 3D y gráficos en auge. Los grandes cálculos en tiempo real con su correspondiente carga para la CPU plantearon dos grandes retos: el avance de los motores físicos cada vez más precisos y potentes, y, por otro lado, la optimización del hardware para llegar a la demanda computacional que esto suponía.

En 2002, se funda Ageia Technologies, responsable de la creación del primer procesador de física dedicado para ordenadores, denominado PhysX PPU (Physics Processing Unit), así como del motor PhysX ³, que sigue siendo líder en la industria a día de hoy. Posteriormente, NVIDIA Corporation, conocida por su papel en la producción de GPUs, adquiere Ageia, impulsando el cálculo de físicas en GPU. Esto, sumado a la dificultad de penetrar en el mercado del hardware, hace que la producción de PPUs quede obsoleta.

Por otro lado, Intel adquiere en 2007 Havok ⁴, otro de los primeros motores de físicas, cancelando el programa HavokFX que pretendía llevar el cálculo de físicas a la GPU para realizar estos en sus CPUs multinúcleo. Posteriormente es adquirido por Microsoft en 2015 y se integra en las herramientas de desarrollo como DirectX. Se establece como el middleware más común en grandes producciones y simulaciones complejas en estudios, mientras que Physx se convierte en libre acceso en 2018 y se integra en motores de videojuegos como Unreal.

La incorporación de motores de física completos en el desarrollo de videojuegos aporta escalabilidad en el *gameplay*, manteniendo la coherencia de sus elementos físicos, lo que resulta imprescindible para la inmersión del jugador Hecker (2000).

¹Según The Big Engines Report 2025 por parte de Sensor Tower https://app.sensortower.com/vgi/assets/reports/The_Big_Game_Engines_Report_of_2025.pdf

²Véase distintas físicas integrables en Unity: <https://docs.unity3d.com/Manual/physics-integrations.html>

³Documentación de Physx: <https://developer.nvidia.com/physx-sdk>

⁴Documentación de Havok: <https://www.havok.com/havok-physics/>

2.3. Simulación física en videojuegos

Hasta la llegada de los motores comentados, existían otras técnicas para representar comportamientos físicos, que eran todavía muy rudimentarias: se diseñaban las interacciones físicas previamente para parecer realistas, ya que dada la capacidad de cómputo con la que solían contar las máquinas, era imposible hacer todos los cálculos físicos en tiempo real. Se realizaban detecciones de colisiones simple, sin uso de fuerzas y descripciones de movimientos o respuestas de cada elemento del juego en lugar de sistemas de física.

Estos métodos eran claramente limitados en la precisión que ofrecían, y quedaba clara la necesidad de una solución más general y reproducible. El mismo Ian Millington introduce en su libro *Game Physics Engine Development* [Millington \(2007\)](#) el juego *Exile*, creado por Peter Irvin y Jeremy Smith; si bien este juego fue publicado en 1988, ya contaba con conservación del momento en colisiones, gravedad, y fuerzas como el viento o explosiones. Resalta como uno de los primeros juegos que cuenta con un motor de físicas completo.

Actualmente existen juegos con simulaciones físicas muy realistas como *Red Dead Redemption 2* [Rockstar Studios \(2018\)](#) o *Hogwarts Legacy* [Avalanche Software \(2023\)](#) donde más allá de satisfacer necesidades de gameplay, se consiguen mundos creíbles en su conjunto. Otros juegos se basan directamente en el funcionamiento y excelencia de sus físicas, particularmente simuladores de vehículos o deformaciones como la saga de *Assetto Corsa* [Kunos Simulazioni \(2014\)](#), el nuevo *Forza Horizon 6 Playground Games (2026)*, *Microsoft Flight Simulator 2024* [Asobo Studio \(2024\)](#) o *BeamNG.drive* [BeamNG \(2015\)](#), que sigue destacando por su física basada en cuerpos blandos.

Se entienden los objetos deformables o cuerpos blandos como aquellos que no son rígidos. Así como en los objetos rígidos puede simularse su movimiento sabiendo su trayectoria y centro de masa, para los objetos deformables se necesita tener en cuenta todas las partículas y propiedades físicas que lo componen. Entre estos se encuentran elementos como las telas, el agua o el viento [Kenny Erleben \(2005\)](#).

Estas simulaciones son posibles gracias al uso de *solvers*. Pese a encontrar múltiples acepciones en el término solver para distintas resoluciones de problemas, aquí nos referiremos como solver al algoritmo computacional que se encarga de predecir el comportamiento de estos objetos deformables en renderizados y simulaciones físicas, específicamente, de telas. Encontramos infinitos tipos de solvers. Los primeros solvers se basan en los algoritmos que emplean la fuerza bruta: realizan cálculos en masa para sistemas de muelles (MSS o Mass Spring Systems) basándose en la Ley de Hooke. Los vértices de la tela tienen un peso y están unidos por tres tipos de muelle que aseguran que la tela mantenga su forma, no se distorsione o doble en exceso ([Provot](#)

et al. (1995)). Los solvers más actuales resultan en el PBD, proceso iterativo que proyecta las posiciones de las partículas sobre un espacio de restricciones (Müller et al. (2007)), es decir, actualizando el desplazamiento generado por cada fuerza en cada vertice de la simulación (Chaudhary et al. (2025)). El solver de PhysX se basa en PBD, es decir, trabaja directamente sobre las posiciones. Encontramos también XPBD (Macklin et al. (2016)), una versión extendida que puede incorporarse en Unity.

Otro concepto de gran importancia para este estudio es el SDF, que será mencionado frecuentemente en los siguientes apartados a la hora de trabajar con las colisiones con objetos externos. Los *Signed Distance Field* representan la distancia entre un punto y la superficie más cercana de un objeto o conjunto de coordenadas. Son frecuentemente usados en la informática gráfica y generalmente se calculan como la distancia ortogonal a un espacio métrico.

2.4. Simulación de telas

El protagonista de la simulación de telas es la vestimenta: vestidos, cintas o capas moviéndose dependiendo de las acciones del jugador, viento y condiciones específicas; lo que permite a los jugadores un mayor grado de inmersión y una mayor credibilidad del mundo del videojuego. Además de tener objetos hechos a partir de tela como ropa o banderas, pueden incluso aplicarse las físicas de la tela en otros elementos como pelo, hierba o cuerpo blandos como globos; como señala Havok: “*Havok Cloth is used to create secondary motion and to enhance any object that flows*”.

Algunos juegos que destacan por comenzar a usar la simulación de telas en tiempo real son *Assasins Creed Unity*(2014) Ubisoft Montreal (2014) que usó un sistema personalizado de Havok Cloth y *Batman:Arkham Knight* (2015) ? con un sistema híbrido de físicas y rigging usando Apex Cloth.

En esta sección se describe cómo se representa y simula una tela en tiempo real, el problema que supone y qué tipos de soluciones encontramos. Se indagará mayoritariamente en ejemplos de Unity3D, como referencia elegida de motor de videojuegos.

2.4.1. Cloth

Cloth⁵ es un componente que proporciona Unity3D que funciona junto al componente Skinned Mesh Renderer para simular telas. Está basado en Cloth de PhysX⁶ y fue incorporado con el objeto de poder disponer de ropa para personajes. Es altamente configurable, lo que le permite adaptarse a multiples tipos de telas. Algunos de los parámetros que permite son: resistencia a la flexión, rigidez, *damping* o uso

⁵Documentación de Cloth (Unity): <https://docs.unity3d.com/Manual/class-Cloth.html>

⁶Documentación de Cloth (Physx): <https://archive.docs.nvidia.com/gameworks/content/gameworkslibrary/physx/guide/3.3.4/Manual/Cloth.html>

de gravedad.

Physx trata la tela como una malla de partículas conectadas por restricciones. El *Skinned Renderer* calcula las transformaciones basándose en una malla de baja resolución sobre la que realiza los movimientos en los vértices, que dinámicamente se aplican en la malla real y se obtiene la simulación visual. Aquellos constraints no visibles o de distancia 0, fijos, no serán calculados. La tela cuenta con capacidad de colisionar internamente y con dos tipos de colliders externos: cápsulas o esferas. Cualquier objeto externo con el que se quiera colisionar tendrá que ser un conjunto de estos colliders.

Los primeros solvers para telas se basan en algoritmos que emplean la fuerza bruta: realizan cálculos en masa para sistemas de muelles (MSS o Mass Spring Systems) basándose en la Ley de Hooke. Los vértices de la tela tienen un peso y están unidos por tres tipos de muelle que aseguran que la tela mantenga su forma, no se distorsione o doble en exceso Provot (1995). Los solvers más actuales resultan en el PBD, proceso iterativo que proyecta las posiciones de las partículas sobre un “espacio de restricciones” Müller et al. (2007), es decir, actualizando el desplazamiento generado por cada fuerza en cada vertice de la simulación Chaudhary et al. (2025).

El solver de PhysX se basa en PBD, es decir, trabaja directamente sobre las posiciones. Asimismo XPBD: Position-Based Simulation of Compliant Constrained Dynamics Macklin et al. (2016), es una versión extendida que con un coste ligeramente superior incide en la alta dependencia sobre el paso del tiempo e iteraciones de la simulación de PBD. Unreal con Chaos integrado ya acepta constraints de tipo XPBD y también puede integrarse en Unity.

2.4.2. Optimizaciones en simulación de telas

Como se ha introducido antes, la simulación de telas en tiempo real supone una serie de desafíos conocidos, de los que se sigue buscando la optimización a día de hoy. El aumento de tamaño o complejidad de una tela, con su alta carga computacional, conlleva el decaimiento inmediato de frames. A parte de dificultar el realismo en juegos y demás simulaciones a tiempo real, resulta un gran impedimento en entornos donde se requieren tasas altas de fotogramas por segundo.

Mientras que el Cloth de PhysX puede delegar la simulación a GPUs compatibles con CUDA, el Cloth integrado por defecto en Unity realiza todo su trabajo en la CPU, lo cual ralentiza mucho la ejecución. En el caso de Unreal, previamente al lanzamiento de ML Cloth Simulation, Chaos Physics delegaba su mayor fuerza de trabajo en la CPU.

Existen varios tipos de soluciones para la optimización de la simulación de telas:

- **Optimización por CPU:** La extensión de *Obi Cloth* ⁷ aprovecha el sistema

⁷Obi en la Unity Asset Store: <https://assetstore.unity.com/packages/tools/physics/>

de Jobs de Unity para paralelizar la simulación y acelerar la simulación sin salir de la CPU. Es uno de los paquetes más populares para solucionar este problema.

- **Paralelización en GPU:** a través de *shaders* se delega la fuerza de cálculo del *skinning* en la GPU. Diferentes estudios aportan diferentes enfoques para la optimización: [Jim Hugunin \(2016\)](#) realiza una investigación para alcanzar altas resoluciones y evitar colisiones en las mallas, mientras que otros se centran en la mejora de rendimiento pura para adaptar simulaciones a entornos VR [Va et al. \(2021\)](#).
- **Entrenamiento con IA:** el enfoque más actual de la resolución de simulaciones complejas como Cloth es la Inteligencia Artificial. Se entrenan modelos de ML con los datos de las mallas animadas o poses, con lo que se obtiene más adelante mejor rendimiento en tiempo real. Se sacrifica memoria y tiempo de entrenamiento para obtener mejores y mucho más rápidas simulaciones de tela en escena. Actualmente la investigación se encuentra en el contexto de implementación para grandes empresas: Unreal ya proporciona una simulación de tela a través de aprendizaje automático, en el que destaca su realismo. Aún así, cuenta con limitaciones al basarse en un *NearestNeighborSearch*: “el sistema es capaz de predecir detalles como arrugas pero no movimiento dinámico como ondulaciones o drapeados”⁸. En la próxima sección se detallarán las numerosas investigaciones recientes que abarcan aspectos desde la reproducción de arrugas, el compromiso entre calidad, coste y rendimiento hasta colisiones y penetraciones en las mallas.

2.5. IA para optimización de telas

Como se ha introducido previamente, en la industria del videojuego, como en muchas otras, la atención está fijada en la inteligencia artificial. Destaca la presencia del ML como optimización de múltiples problemas físicos, en este caso, de telas y cuerpos no rígidos similares. En este apartado se presentan los recursos e investigaciones más relevantes para el desarrollo de este proyecto.

2.5.1. Modelos de optimización de telas

Desde la introducción del PBD, la optimización de simulaciones con redes neuronales cuenta ya con un amplio recorrido. En este apartado se describen los estudios más destacados de los últimos años. Son también inspiración directa para la implementación de nuestros experimentos, con el objetivo de descubrir cuáles de estas técnicas son verdaderamente aplicables a menor escala, en desarrollo indie con recursos limitados en cuanto a hardware y especialización físico-matemática.

obi-cloth-81333?aid=1011134eQ

⁸Documentación introductoria de Chaos Cloth: <https://dev.epicgames.com/documentation/unreal-engine/machine-learning-cloth-simulation-overview>

2.5.1.1. Subspace Neural Physics: Fast Data-Driven Interactive Simulation

Este desarrollo de la unidad de investigación Ubisoft La Forge [Holden et al. \(2019\)](#) implementa la primera técnica que incorpora reducción de modelo o el uso de subespacios con colisiones con objetos externos en la tela. El estudio se centra en conseguir un balance entre el rendimiento en tiempo real y el coste y memoria necesarios para realizar el precómputo.

El entrenamiento se realiza sobre las posiciones de una simulación realizada en Maya aplanados en un vector, al que se añade otro vector plano con los parámetros necesarios como posición del suelo, valor del viento, o collisionadores con la tela. Se aplica un PCA⁹ sobre ambos denotando cuántos ejes de deformación de la tela se tienen en cuenta para representar la complejidad de la recreación: 64, 128, 256; siendo este último el que conserva casi el mismo nivel de detalle que la simulación original.

Para evitar que la simulación colapse con el error acumulado aplican dos estrategias: ventanas de frames superpuestas, con bloques de 32 frames con mini-batches para mantener la coherencia a largo plazo, y ruido gaussiano para que la red aprenda a corregirse. Además, los cálculos necesarios durante la ejecución se aceleran con shaders en GPU (GPU decompression) y se recalculan las normales de los vértices necesarias para el renderizado.

Sus principales limitaciones residen en la necesidad de quitar a mano errores producidos en la simulación, falta de pruebas en objetos plásticos, posibles colisiones internas fallidas, y limitación de objeto colisionador fijo en el espacio y sin contar con objetos cayendo por el espacio. Se necesitaron 10^6 frames de simulación para entrenamiento para mejores resultados, así como períodos de entre 10 y 48 horas de entrenamiento.

2.5.1.2. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network

Este estudio [Chentanez et al. \(2020\)](#) es un ejemplo de uso de CNN para la simulación de telas. Reseña la dificultad de integrar una CNN debido a la construcción de la malla, incompatible antes de su modificación con una cuadrícula 2D. Resuelve el problema entrenando mallas "manifold" triangulares: es decir, mallas cuyos bordes están conectados por al menos dos caras (menos los extremos) dejando una separación entre exterior e interior. Mapear la malla a una imagen 2D es lo que permite realizar la convolución.

⁹*Principal Component Analysis*, es una técnica que disminuye el número de variables de un dataset, preservando la mayor cantidad de información y varianza posible y reduciendo así su complejidad

Crean una red encoder-decoder¹⁰ formada por bloques “DownConv”, para la reducción, y “UpConv”, para la reconstrucción, que integra operadores nuevos de convolución¹¹. En el bloque de “DownConv”, utiliza una técnica de pooling destacable: colapasa los edges siguiendo un método en espiral. También señala la importancia de usar Leaky-CNN: para no dejar neuronas muertas, que resulta un problema en la simulación. Además de combinar funciones de pérdida L2 y L1¹², para que no se quede el error acumulado en ciertos vértices.

Este estudio, a diferencia del anterior, incluye brazos y manos, y se rige en un marco de posiciones locales por cada triángulo. Afirma conseguir resultados muy cercanos al *ground truth* usando la base de animaciones *Berkley animation base* y una gran capacidad de hardware disponible. No obstante, resaltan una vez más los largos tiempos de entrenamiento (entre 4 y 20 horas) y la necesidad de calidad y cantidad de datos de entrenamiento. Tampoco aporta simulaciones para topología dinámica y se ciñe mucho al entrenamiento.

Otro ejemplo de estudio que usa una CNN para la simulación de telas es DeepSD: Real-time Cloth Simulation with Deep Learning Bertiche et al. (2021b); en la que proyecta la malla 3D en un plano utilizando sus coordenadas UV. Usa una CNN estándar, y no depende de una malla fija, pudiendo aceptar cambios en la topología de la malla, por lo que sirve para automatizar el proceso. El uso de SDF garantiza la generación de ropa que envuelva el cuerpo sin importar la conectividad de la malla durante los cálculos intermedios. No obstante, comparando con el estudio de Chentanez et al. este estudio no consigue capturar arrugas finas o superficiales con detalle y es menos eficiente debido a su enfoque volumétrico.

2.5.1.3. PBNS: Physically Based Neural Simulation for Unsupervised Garment Pose Space Deformation

PNBS Bertiche et al. (2021a) presenta una simulación neuronal DL no supervisada, como alternativa al enfoque clásico PBS. Integra principios físicos en el entrenamientos con funciones físicas de energía, entre ellas restricciones de fijación. La arquitectura consiste en un perceptron multicapa (MLP, por sus siglas en inglés *Multi-Layer Perceptron*) con funciones ReLU que genera deformaciones tipo PSD sobre modelos LBS, cuya aplicación en el mundo 3D garantiza la compatibilidad con los motores gráficos y su alta eficiencia. Esta metodología entrena sobre data-sets públicos como *AMASS* Mahmood et al. (2019) y *CMU MoCap*, siendo capaz de

¹⁰**Red encoder-decoder:** Arquitectura de red neuronal diseñada para mapear una secuencia de entrada a una secuencia de salida, donde ambas pueden tener longitudes diferentes. El *Encoder* comprime la información de entrada para operar con ella fácilmente, y el *Decoder* procesa esa representación para generar la salida con las dimensiones reales.

¹¹**Convolución:** Operación matemática donde un filtro o *kernel* realiza multiplicaciones elemento a elemento, permitiendo extraer patrones estructurales.

¹²**L2 y L1:** Funciones que miden la diferencia entre el valor real y el predicho. La pérdida L_1 (Error Absoluto Medio) calcula la suma de las diferencias absolutas, siendo robusta frente a valores atípicos. La pérdida L_2 (Error Cuadrático Medio) calcula la suma de los cuadrados de las diferencias, penalizando fuertemente los errores grandes.

manejar colisiones en poses extremas, así como colisiones cloth-to-cloth. Asimismo, permite la obtención de prendas que se ajustan a diferentes cuerpos (garment resizing). En comparación a otros métodos como TailorNet [Patel et al. \(2020\)](#), logra una mayor consistencia física y generalización, además de un coste computacional considerablemente menor, incluso sin GPU. Sin embargo, se ve limitado por su enfoque en poses estáticas.

2.5.1.4. SNUG

Siguiendo la tendencia hacia el aprendizaje no supervisado basado en físicas, la técnica SNUG [Santesteban et al. \(2022\)](#) va más allá de la deformación basada en poses mencionado en el estudio anterior. Si bien trabaja con 6,519 fotogramas de la misma base de datos, gracias a una red recurrente (cuya arquitectura no se explica con exactitud) consigue mejores tiempos de entrenamiento y resultados más detallados que la técnica anterior. Pone el foco del aprendizaje crear una serie de funciones de loss basadas en las físicas reales de deformaciones dinámicas: “Membrane Strain Energy” (la energía transmitida entre dos puntos en base a la elasticidad del material), “Bending Energy” (el ángulo diédrico entre dos caras en relación a su propiedad de rigidez), “Gravity” (gravedad) y “Collision Penalty” (la distancia prudencial entre el modelo y el cuerpo).

2.5.1.5. Neural Cloth Simulation

Neural Cloth Simulation [Bertiche et al. \(2022\)](#) también propone una metodología no supervisada, unificando simulación y entrenamiento. Sigue el esquema clásico de este tipo de solvers, donde el estado de la tela depende de los instantes previos, gestionado por ventanas de frames cuyo tamaño y complejidad dependen del tipo de tela procesada. Su arquitectura consiste en un encoder-decoder recurrente que separa componentes estáticos y dinámicos, haciendo uso de modelos GRU en estos últimos y la cual permite mejorar la generalización y estabilidad del entrenamiento. Este se realiza mediante funciones de pérdida inspiradas en simulación física, incluyendo restricciones físicas en relación a deformaciones, colisiones e inercia. El dataset utilizado también es AMASS [Mahmood et al. \(2019\)](#), que recopila información de mas de 40.000 poses, y el modelo físico extiende la idea del modelo mass-spring [Provot et al. \(1995\)](#). En comparación con PBNS o SNUG, esta metodología consigue soluciones más robustas y simples que dan a simulaciones detalladas a costa de mayores tiempo de entrenamiento que sus predecesores, compensando, no obstante, la disminución de recursos al no realizar procesos de generación de datos. Esta perspectiva va de la mano con tendencias actuales en la industria, como la ya mencionada e implementada Chaos Cloth de Unreal Engine.

2.5.1.6. HOOD: Hierarchical Graphs for Generalized Modelling of Clothing Dynamics

En investigaciones más recientes, se aborda la cuestión de la posibilidad de abstraer también la topología de la tela del aprendizaje. En este caso, HOOD utiliza GNNs, una arquitectura de envío de mensajes jerárquico y por supuesto la función

de pérdida basada en la física que puede verse en los previos estudios Grigorev et al. (2023). Los graph neural networks permiten crear un gráfico que incorpora los vértices del cuerpo colisionador. A través de este se propagan los mensajes de aceleración recursivamente, lo que permite ajustar la precisión de la inferencia y adaptarse a distintos materiales, topologías y modelos de colisión.

Este estudio sigue estando limitado en aspectos como la velocidad de simulación o colisiones internas. Al fin y al cabo, este sigue siendo un campo de estudio en crecimiento, en el que todavía queda mucho por avanzar.

2.5.2. Librerías

Por último, en este apartado se describen las librerías que han resultado indispensables para entrenar los modelos de IA y conectarlos a la simulación en Unity.

2.5.2.1. Pytorch

Pytorch es una de las librerías de código abierto que permite implementar redes neuronales propias. Los tensores¹³, matrices multi-dimensionales análogas a los NumPy arrays de Python, permiten optimizar las redes tanto para el uso de CPU como de GPU¹⁴. Suele ser la más elegida en entornos académicos, ya que permite prototipado rápido y entrenamiento dinámico. Fue originalmente desarrollado por Meta, y todavía cuenta con una enorme comunidad de desarrolladores.

2.5.2.2. ONNX

ONNX es un formato creado para representar modelos de aprendizaje automático¹⁵. Fue introducido por Facebook y Microsoft en 2017, con el objetivo de facilitar el cambio de entornos de trabajo en aprendizaje automático. Estandariza la estructura de redes neuronales y tipos de datos, definiendo un conjunto común de operadores y un formato de archivo estándar, permitiendo entrenar desplegar modelos entrenados en un distintos ecosistemas. Es un formato ampliamente aceptado por fabricantes de hardware como NVIDIA o Intel, que proporcionan proveedores de ejecución especializados para la conexión.

2.5.2.3. Sentis

Sentis (previamente Inference Engine) es una librería de inferencia de redes neuronales para Unity, predecesora introducida en el 2023 de la librería Barracuda. Permite importar modelos de redes neuronales entrenados a Unity y ejecutarlos en tiempo real tanto con la CPU como con la GPU¹⁶ de forma local en el usuario,

¹³Documentación correspondiente a los tensores: <https://docs.pytorch.org/docs/2.12/tensors.html>

¹⁴Documentación de Pytorch: <https://docs.pytorch.org/docs/stable/index.html>

¹⁵Página web de ONNX: <https://onnx.ai/>

¹⁶Documentación del paquete Sentis: <https://docs.unity3d.com/Packages/com.unity.ai.inference@2.4/manual/index.html>

reduciendo latencia y costes de servidores en la nube.

Los modelos pueden ejecutarse localmente en las plataformas que soportan Unity, lo que permite que pueda adaptarse a diferentes tipos de hardware: tanto en PC, como en móviles, consolas o web. Su compatibilidad reside en los modelos importados en formato ONNX, cuyos grafos se optimizan después para la ejecución del juego.

Algunos usos de Sentis son las estimaciones de poses en tiempo real o NPCs que reaccionen a texto o voz del jugador.

Descripción del Trabajo

3.1. Arquitectura General

El desarrollo de las pruebas de este proyecto se divide en tres secciones; la de simulación, la de entrenamiento del modelo y la de inferencia. En este apartado se explican estas de manera general, para crear una base de entendimiento del pipeline. En los proximos apartados del capítulo se explica cada elemento de manera más exhaustiva, con las distintas iteraciones y mejoras conseguidas a raíz de la investigación.

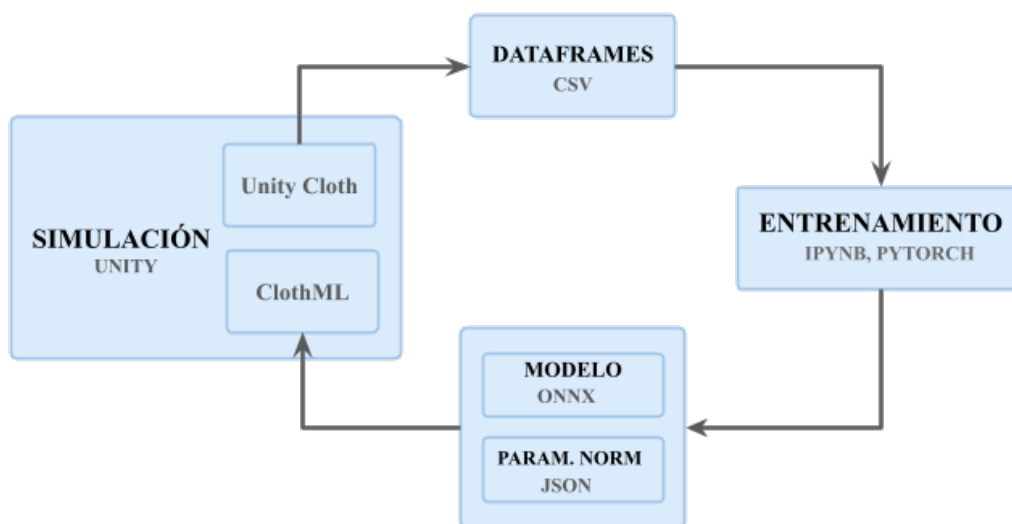


Figura 3.1: Diagrama de la arquitectura General del proyecto

Como puede apreciarse en la figura 3.1, el pipeline del proyecto es circular, es decir, la plataforma en la que se genera la simulación y posteriormente se prueba la inferencia del modelo es la misma; en nuestro caso, Unity. Esto se debe a dos de nuestros objetivos: el de hacer un sistema accesible y fácil de utilizar y el de poder

comparar verazmente la optimización de la simulación física.

- **Simulación:** Si bien se han llevado a cabo puebas con distintas geometrías y tiempos, colisionadores con distintos tipos de movimiento y aleatoriedad de los mismos, las bases de las escenas es equivalente. La simulación, en el que la tela actúa con componente Cloth de Unity, pasa por un sistema de registro con el que se crea un archivo CSV que recoge la información de la geometría simulada y las variables pertinentes.
- **Entrenamiento:** Estos archivos se recogen en una carpeta que pasa a procesarse en un entorno de python. Con las librerías explicadas en el estado del arte se han entrenado distintos modelos, iterando sobre ellos hasta conseguir las configuraciones e hiperparámetros óptimos. La efectividad de estos modelos se mide en primer lugar dentro del entorno de python con el “accuracy” de las predicciones, midiendo el error de distancia real de los vértices, visualizando la progresión a través de distintos frames.
- **Inferencia:** Posteriormente se exportan estos modelos ya entrenados en formato ONNX a la escena de Unity en la que se hizo la simulación inicial y correspondiente recogida de datos. Haciendo uso de la arquitectura dirigida a objetos de Unity, se cambia la configuración de la geometría inicial y se incorpora el modelo entrenado y sus parámetros, consiguiendo así inferir el movimiento de la tela en tiempo real y confirmar que la simulación es visualmente fiel a las físicas reales.

3.2. El dataset

Para poder entrenar un modelo de Inteligencia Artificial que recree el movimiento de una tela se necesita un dataset que represente el comportamiento de esta.

3.2.1. La simulación

Los datasets son generados de forma interna, creándose a partir de simulaciones de telas grabadas en Unity.

La simulación base que se emplea para entrenar y ajustar los modelos se lleva a cabo en un escenario formado por un plano-tela de 25 vértices con la parte superior fija en el espacio, y una esfera que colisiona con la tela para generar el movimiento.

Durante parte del desarrollo del modelo, para poder identificar los motivos de inestabilidades e identificar otros errores con más facilidad, se redujo el tamaño de la malla a 6 vértices.

El movimiento de la bola que colisiona con la tela está descrito por un componente *BallMovement*. Este puede ser automático o generado a través de entrada con el teclado. El movimiento automático tiene dos movimientos descritos:

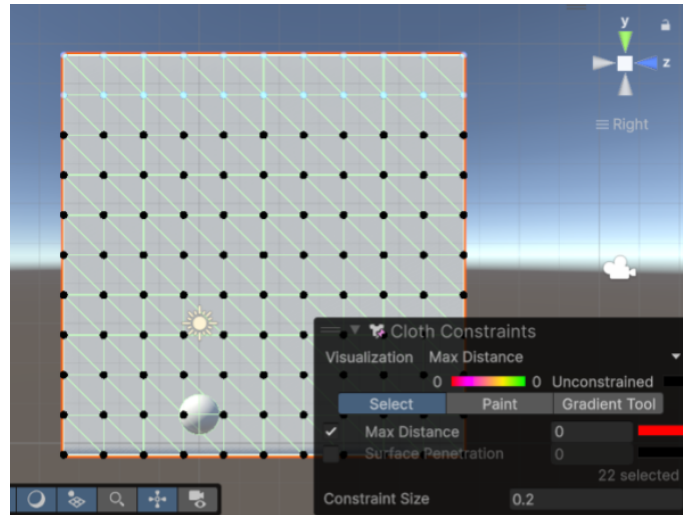


Figura 3.2: Malla de 25 vértices con parte superior fija.

- **Movimiento lineal:** ideado para provocar un movimiento sencillo que entienda el modelo fácilmente, sin deformaciones extremas o erráticas, y utilizado para poder llevar a cabo overfitting. Se consigue estabilizar el modelo sobre un movimiento simple. El movimiento consiste en oscilar entre dos posiciones fijas en el espacio a lo largo del eje x.
- **Movimiento aleatorio:** pretende facilitar la generación de un dataset con diversidad de movimientos de forma automática, así como introducir a la tela más dinamismo y movimientos erráticos. Este desplazamiento en tres ejes aleatorio se consigue a través del uso de splines. En la escena se encuentra un objeto con el componente creado *PathGenerator* que delimita una zona de generación y aporta la creación de splines en *SplineContainers* a partir de una posición concreta. El componente de Unity *SplineAnimate* se encargará de mover la esfera por el camino hasta llegar al final del mismo; en ese instante se generará un nuevo camino aleatorio para continuar con su recorrido.

Adicionalmente, introducimos modificaciones sobre este comportamiento: se deja el colisionador estático en distintas posiciones tanto dentro como fuera del alcance de la tela durante periodos de tiempo en la ejecución para intentar limitar la dependencia entre la tela y su distancia a la esfera.

3.2.2. Otras simulaciones

También se han realizado pruebas con simulaciones más complejas con objeto de probar la efectividad de nuestro método y diferencias entre simulaciones más dinámicas y estáticas, más frecuentadas en PBD.

Para probar el enfoque más estático y pegado al cuerpo se creó una simulación de un personaje con una **camiseta**. Tras múltiples pruebas tomando diferente número de iteraciones en el solver de Cloth, valores en las propiedades de la tela y extendido

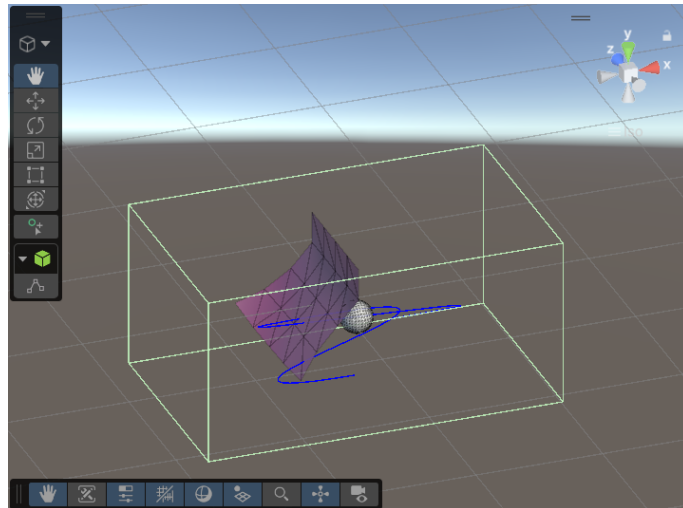


Figura 3.3: Movimiento aleatorio basado en splines.

el tamaño hasta 4000 vértices, la simulación fue finalmente descartada debido a la inestabilidad de la tela y difícil generación de animaciones visualmente coherentes. Realizar animaciones de tela complejas en Unity sin combinar otras técnicas, implementar soluciones híbridas (como el Weight Mapping¹) o incorporar paquetes de pago de la Unity Asset Store como Magica Cloth² u Obi Cloth no es trivial. Otras investigaciones, como Holden et al. (2019) producen su material de entrenamiento en herramientas especializadas como Maya³ o Blender⁴ para conseguir simulaciones de prendas complejas. Pese a que esto limita el alcance de las simulaciones a generar en esta investigación, el nivel de complejidad potencial sigue siendo más que suficiente para probar nuestra implementación a pequeña escala, manteniendo el proyecto alienado a los objetivos previamente declarado y siguiendo la tendencia de juegos implementados en Unity.

¹Técnica de animación 3D y rigging que permite controlar cómo se deforman los vértices de una malla o prenda al mover las articulaciones de un esqueleto.

²Paquete de Magica Cloth en el Asset Store de Unity: <https://assetstore.unity.com/packages/tools/physics/magica-cloth-2-242307>

³Autodesk Maya es un software profesional de animación y efectos visuales 3D <https://www.autodesk.com/es/products/maya/overview>

⁴Blender es un software de código abierto de animación y efectos visuales 3D <https://www.blender.org/>



Figura 3.4: Simulación de una camiseta con componente Cloth.

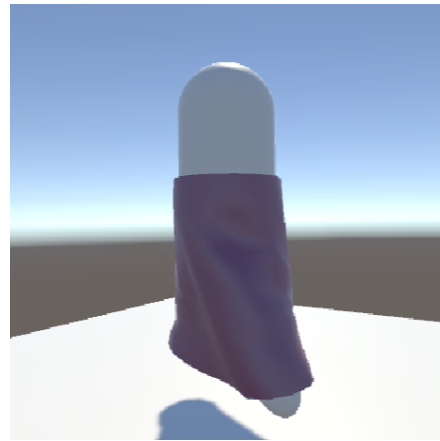


Figura 3.5: Simulación de una falda con componente Cloth.

La otra simulación llevada a cabo consiste en un personaje andando con una **falda**. El personaje, simplificado, está formado por tres cápsulas: un torso y dos piernas; y la falda es un malla en forma de cilindro compuesta por 288 vértices, para habilitar un movimiento más dinámico. Una vez ajustados los colliders de las cápsulas para ser más grandes que las mallas de las piernas, esta simulación genera un buen resultado a nivel visual. Lo emplearemos más adelante para representar animaciones más complejas y realizar comparaciones y mediciones adicionales al plano de 25 vértices.

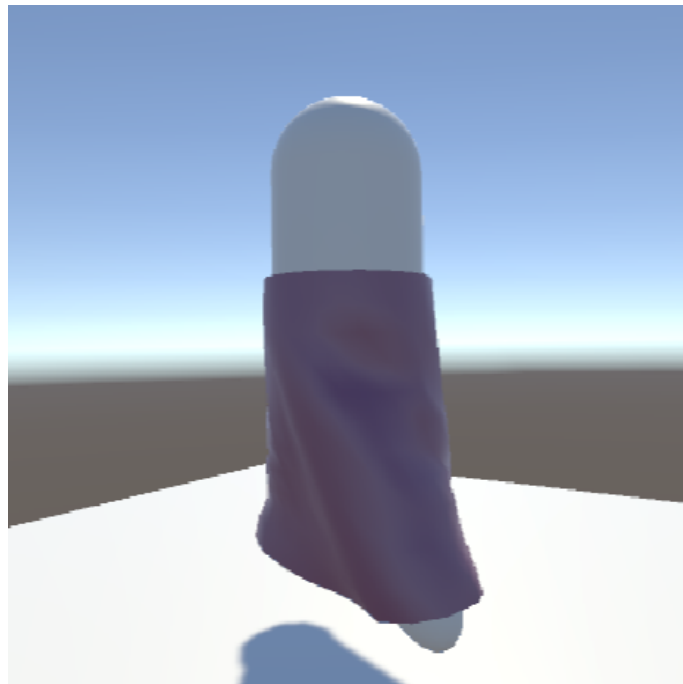


Figura 3.6: Simulación de una falda con componente Cloth.

3.2.3. Los parámetros

Las telas que usamos para generar las simulaciones en las que se basarán nuestros modelos son objetos formados por mallas con un componente Cloth (y su correspondiente *Skinned Mesh Renderer*) y un componente de grabación *ClothDatasetRecorder*, encargado de guardar los datos de la simulación. Se plantea recoger diferentes parámetros que representen las características más importantes de la tela durante la simulación:

- **Posición:** Las posiciones de los vértices de la tela son los parámetros básicos de las investigaciones en este campo. Estas posiciones, se verá más adelante, pueden darse en espacio local del objeto o global.
- **SDF:** El Signed Distance Field es una magnitud que representará la distancia de cada vértice al objeto colisionador, llegando a ser 0 si toca la superficie, y un valor negativo si alcanza a traspasar. Se ha creado la clase estática *SDFUtil* para habilitar los cálculos del SDF para conjuntos de *colliders*. El script *SDFUtil* contiene métodos que proporcionan el SDF de un punto a una esfera, una cápsula o dos listas de los dos tipos. También acepta un *Transform* de un objeto de referencia si la posición del punto de origen no es global.

Las fórmulas de SDF, así como la obtención de un único SDF respecto a varios objetos, que se ha decidido representar con el *Smooth Minimum* (*smín*), se han adquirido del blog de Inigo Quilez⁵ y adaptado para el funcionamiento esperado en Unity. La función *smín* permite fusionar dos objetos de geométricamente con una transición que puede ser suave o limitarse al espacio real ocupado por los objetos.

- **Velocidad:** Planteamos que las velocidades de los vértices pueden ayudar a que el modelo sea capaz de replicar el dinamismo del movimiento.
- **Normales:** Refiriéndose a las normales de la superficie del objeto colisionador, es decir, el gradiente del SDF. Este se puede usar para complementar la magnitud del SDF, ya que apunta en la dirección en la que la distancia aumenta más rápido, cómo alejarse de la colisión.
- **UVs:** Conservar la geometría de la tela se plantea como uno de los mayores retos para nuestra simulación. Se necesita que el modelo comprenda cómo están relacionados los vértices espacialmente para que no se lleven a cabo deformaciones erróneas. Ya que no comenzamos con geometrías complejas en tela o cuerpos blandos, se usan las coordenadas de textura bidimensionales, *UVs*, para representar la superficie.
- **Máxima Distancia:** Cada vértice tiene una distancia máxima a la que un vértice se puede mover desde su posición en la malla *skinned*⁶. Se quiere con-

⁵Apartado del blog de Inigo Quilez sobre SDFs para objetos 3Ds: <https://iquilezles.org/articles/distfunctions/>

⁶Una *skinned mesh* es una malla poligonal 3D cuyos vértices están unidos a un esqueleto jerárquico de huesos, que permite la deformación dinámica de sus vértices.

seguir que los vértices no se pasen de su rango hábil de movimiento. Si no se toma como parámetro, esto puede corregirse también en post-procesamiento.

3.2.4. Generación del dataset

A través del componente *ClothDatasetRecorder* realizamos un “snapshot” o captura de la tela cada intervalo de tiempo *recordTime*, que se declara en el inicio del desarrollo como 3.5 segundos. Estos datos del frame se guardan en una estructura *SnapInfo*, que representa la información de los vértices de la tela en un instante de tiempo determinado, y se añade a una lista de capturas de la simulación. Al final de la ejecución todas estas capturas se transforman en texto y se vuelcan en un CSV, por lo que cada línea del archivo corresponde a un frame capturado.

Algorithm 1 Struct SnapInfo

```
Data: SnapInfo {
    pos: array de Vector3 ;                ▷ posiciones
    vel: array de Vector3 ;                ▷ velocidades
    normal: array de Vector3 ;              ▷ normales
    sdf: array de float ;                  ▷ signed distance field
    maxDist: array de float ;              ▷ distancia máxima
    uv: array de Vector2 ;                 ▷ coordenadas UV
}
```

Durante el transcurso de las pruebas se probaron distintas frecuencias de muestreo (0.1f, 0.02f, la frecuencia por defecto del *FixedUpdate()* en Unity); más tarde se llegaría a la conclusión de que la frecuencia de muestreo debe ser la misma en la etapa de guardado de datos y ejecución del modelo para no obtener un efecto de aceleración o ralentización. De la misma forma, el número de snapshots debe ser el máximo posible para garantizar la estabilidad de la simulación.

3.3. Entrenamiento

En este apartado se describen las fases de entrenamiento de los modelos, así como la evolución de los mismos y la serie de decisiones tomadas para conseguir esas adaptaciones. El código al que se hace referencia puede encontrarse en el directorio /Entrenamiento del repositorio de GitHub del proyecto ⁷.

3.3.1. El entorno de trabajo

Para crear un entorno de trabajo adaptable, y asegurar la consistencia entre los distintos equipos utilizados por las integrantes de este proyecto, se ha decidido crear en el repositorio un archivo *dl2024_gpu.yml* en el que concretar las dependencias

⁷Enlace a la carpeta de entrenamiento: https://github.com/mikeletzu/TFG_FisicaOptim/tree/23105b6733caca9c708593f2d77a12faa2eb884/Entrenamiento

del proyecto y las versiones a utilizar de estas librerías. Este es un tipo de archivo de configuración que permite, entre otras cosas, crear fácilmente Conda Environments. Estos entornos virtuales se generan en un directorio con todas las dependencias especificadas. Se ha utilizado como base el entorno proporcionado por la Universidad de Amsterdam en sus cursos introductorios de Deep Learning con Pytorch.

Librerías utilizadas	Versiones
python	3.12.7
cmake	-
notebook	7.2.2
jupyterlab	4.2.5
matplotlib	3.9.2
scikit-learn	-
pytorch-cuda	11.8
pytorch	2.5.0
onnx	1.21.0
onnxruntime	1.24.2
onnxscript	0.5.6
skl2onnx	1.20.0

Tabla 3.1: Librerías y versiones del entorno de desarrollo.

3.3.2. Apartados de la ejecución del entrenamiento

Dada su flexibilidad para segmentar la ejecución y su capacidad de visualizar los resultados claramente, se ha elegido IPython Notebook (.ipynb) como formato de código en el que trabajar con los modelos. Si bien cada tipo de modelo se ha creado en un archivo distinto, todos comparten la misma estructura. Pese a la inevitable repetición de código y potencial optimización, la utilización de diferentes notebooks ha resultado clave para el desarrollo e investigación paralela de los diferentes modelos. La estructura tiene la siguiente forma:

1. **Carga y procesamiento de datos:** Los datos se leen del archivo .csv utilizando la librería pandas, que permite llevar a cabo la limpieza fácilmente. Aunque es el dataset del modelo quien gestiona detalles como la coherencia de frames otorgados, intentar acceder al predecesor de un primer frame, se realiza una limpieza común previa del csv:
 - Se elimina la columna "frame" de la tabla, ya que no aporta información pertinente para el aprendizaje. Su función es meramente auxiliar para depurar.
 - Asimismo, en los casos que se explicarán más adelante, muchos de los parámetros registrados no son necesarios, se filtran esas columnas para no sobrecargar innecesariamente el dataset.

Tras hacer la limpieza de los datos estos se transforman al formato de tensores de Pytorch.

2. **Dataset de Pytorch:** Pytorch ofrece dos clases base con las que ordenar y gestionar los datos que requiere el modelo. *Torch.utils.data.Dataset* almacena las muestras y sus etiquetas correspondientes, y *torch.utils.data.DataLoader* permite un fácil acceso a las muestras. En este bloque se definen las características del Dataset en base a las necesidades del modelo; este recibe en su constructora el tensor con la información filtrada y el tamaño de los lotes en los que dividirlo.
3. **Normalización:** Se dividen los datos entre el entrenamiento y las pruebas, generalmente en un ratio de 80/20 respectivamente. Se itera sobre los datos de entrenamiento para calcular los deltas (desplazamiento de movimiento de cada vértice en un frame) y se normaliza tanto el input completo como los deltas generados. La normal y la desviación típica tanto del input como del target se guardan en un archivo de formato JSON *cloth_norm_params.json*, para utilizarlo posteriormente en la ejecución en Unity.
4. **Modelo de Pytorch:** Pytorch ofrece también una clase base a partir de la cual implementar modelos de redes neuronales, *torch.nn.Module*. En este apartado se extiende esta clase a las especificaciones deseadas y se genera utilizando el DataLoader mencionado un primer batch de prueba.
5. **Entrenamiento:** En este apartado se determinan los parámetros del entrenamiento: *learning rate*, iteraciones, porcentaje de ruido y etc. El entrenamiento consiste en una fase de entrenamiento y otra de testing, en la que se observa el aprendizaje del modelo a través de la caída del error medio de la posición de cada vértice.
6. **Visualización de resultados:** Al trabajar con una simulación física en un entorno 3D, la visualización en gráficos de las predicciones de los modelos ha resultado notablemente útil. En las figuras 3.7, 3.8 y 3.9 pueden apreciarse los gráficos más destacados que más han contribuido al desarrollo.

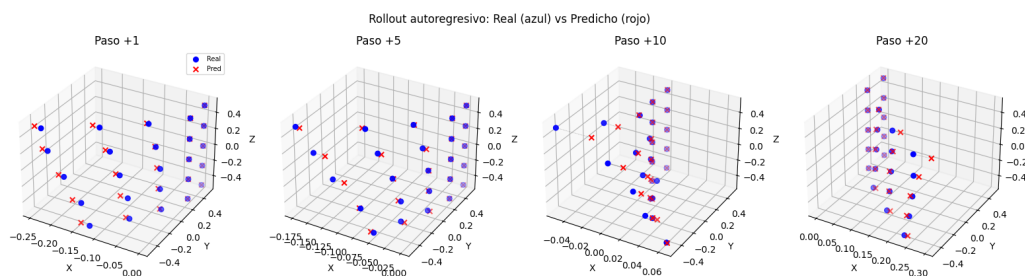


Figura 3.7: Visualización de los puntos de la tela simulada e inferida a lo largo del tiempo

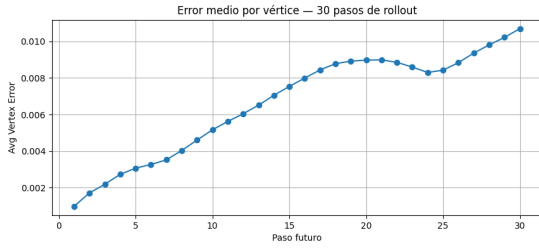


Figura 3.8: Visualización del error medio a lo largo del tiempo

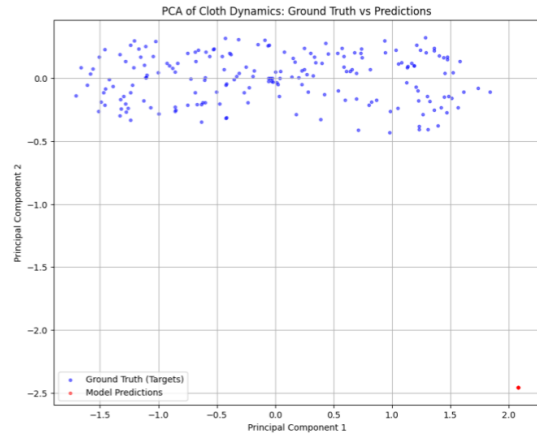


Figura 3.9: Visualización de error de normalización

7. **Exportación:** Por último, tras considerar los resultados generados como adecuados con la guía de los valores y gráficas generadas, se procede a exportar la red neuronal a formato ONNX.

3.3.3. Los modelos

Como se ha mencionado previamente, hay dos nociones principales que han guiado el desarrollo de este proyecto. Por un lado, combinar las técnicas óptimas o prometedoras de las investigaciones citadas en el estado del arte. Y por otro, el objetivo de priorizar la simplicidad y accesibilidad de nuestro modelo. En virtud de ello se optó por empezar el entrenamiento con una red neuronal muy sencilla y aumentar progresivamente la complejidad de la red en base a los resultados, problemas y limitaciones que fueran surgiendo.

3.3.3.1. Modelo inicial

El modelo inicial es un perceptrón multicapa (MLP) sencillo con una sola capa oculta. Este infiere la diferencia de desplazamiento entre un frame y el siguiente. Utilizando un dataset que genera lotes que contienen pares de la información de un frame y su frame consecutivo, se pasa el frame normalizado al modelo y este predice el delta de desplazamiento para cada vértice. Desnormalizando este delta se calcula la posición inferida por el modelo. La diferencia entre esta posición predicha y la posición real recibida en el frameInfo + 1 se utiliza como pérdida para el entrenamiento. La función de pérdida elegida inicialmente, MSELoss, se utiliza generalmente en problemas de regresión pues penaliza fuertemente las diferencias grandes entre la predicción y el valor real, alineándose con nuestro objetivo. El modelo está compuesto por dos capas lineales y una RELU; y sus dimensiones son: 13 de entrada (los parámetros de cada punto), el número de neuronas en la capa oculta (inicialmente entre 64 y 128) y 3 de salida (el delta de desplazamiento en cada eje).

3.3.3.2. Adaptaciones del modelo

El entrenamiento de este modelo recalca ciertas necesidades, algunas más obvias como ampliar el tamaño de la red neuronal y otras menos evidentes que se recogen a continuación:

- **Sistema de coordenadas:** El registro de las posiciones al generar el dataset puede hacerse tanto de manera local respecto al pivote de la tela, como de manera global respecto al espacio 3D de la escena. Al registrar los elementos como las posiciones de los vértices o el SDF se hace de manera consistente, manteniendo un origen común para las coordenadas, por lo que ambos métodos son funcionales. Sin embargo, tras evaluar el desempeño del modelo con datasets relativos a ambos sistemas de coordenadas, se decide trabajar en espacio local, utilizando como origen la propia tela. De esa manera, las posiciones son más coherentes para el modelo y el resultado no es dependiente de la traslación en el espacio.
- **Frecuencia de muestreo:** Inicialmente el dataset se entrena con una serie de frames que se registran en intervalos predefinidos *recordTime* a la hora de hacer la simulación base. Este método agiliza la tarea de generación del dataset y podría pensarse que simplifica la simulación para el modelo. Al contrario, al disminuir el tiempo entre capturas de frames se aprecia una mejora significativa en el entrenamiento del modelo. Se concluye que el modelo aprende idóneamente con la máxima cantidad de frames posibles. Esta afirmación es respaldada por estudios como [Santesteban et al. \(2022\)](#) en los que se utilizan grandes datasets con secuencias de animaciones completas.
- **Recurrencia:** Varias de las investigaciones del estado del arte, como por ejemplo [Santesteban et al. \(2022\)](#), mencionan como limitación las telas más dinámicas, ya que, al tener muchas menos restricciones, las mallas que no van pegadas a un cuerpo de colisión son mucho más susceptibles a la inercia. Naturalmente, el aprendizaje de estas propiedades físicas requiere del conocimiento de atributos como la velocidad o la aceleración de las partículas. Si bien se tomaba la velocidad como input, la recurrencia permite al modelo tener una memoria a corto plazo del movimiento en cada vértice, haciendo su entrenamiento mucho más eficaz. En este caso se presenta como opción una red neuronal recurrente (RNN) que sigue la arquitectura de tipo "many to one", es decir, una cantidad determinada de frames se utilizan como input para generar el delta de desplazamiento entre el último frame de la secuencia y el que sería el siguiente.

3.3.4. Parametrización

En el apartado del Dataset se han enumerado todas las características de la tela que se guardan en la simulación. Aunque se registran todas para poder disponer de un margen de acción en el entrenamiento, a lo largo del proyecto se han utilizado distintas técnicas para reducirlas y determinar cuales de ellas son esenciales.

- Con objeto de simplificar y reducir errores, se reduce la cantidad de features utilizados al mínimo: la posición de cada vértice y el sdf. Se observa que para mallas sencillas (por ejemplo la anteriormente mencionada malla reducida a seis vértices) y movimientos limitados es un método con bajo error. Además, la poca cantidad de parámetros reduce la duración del entrenamiento y optimiza su incorporación a Unity, de la que se hablará más en la próxima sección.
- Tras confirmar que la reducción de features puede ser interesante, se implementa un modelo *Random Forest* para ver cuáles son los parámetros más importantes para la inferencia. Se mide la importancia de Gini, esta métrica mide la reducción media de la impureza (MDI por sus siglas en inglés: Mean Decrease in Impurity), es decir, cuánta varianza o error reduce cada variable a lo largo de las divisiones en los nodos de todos los árboles. Sumando la importancia acumulada sobre los ejes temporales y espaciales (vértices de la malla), se consigue obtener la importancia de cada una de las 13 propiedades del dataset. Como se ve en la figura, los parámetros U, V y MD resultan no ser importantes para generar la predicción. Sin embargo, se aprecia la importancia de los parámetros de velocidad. En este caso es más notable en el eje x, debido a la configuración de la simulación con la que se ha generado el dataset analizado para el gráfico (3.10).

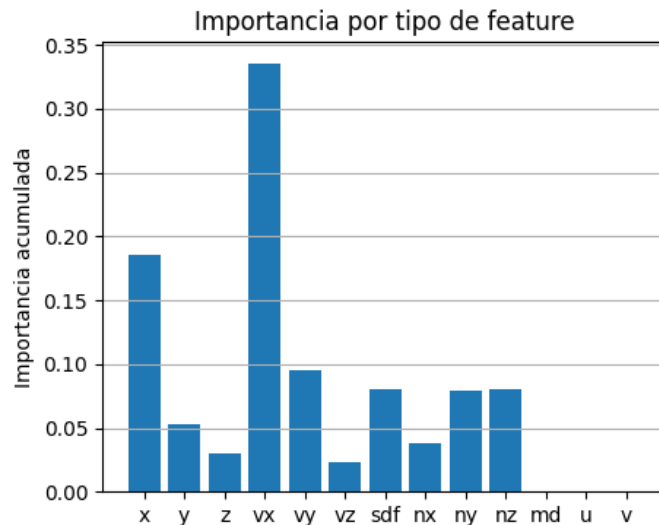


Figura 3.10: Parámetros valorados por el random forest

Con esta prueba se decide incluir la velocidad como parámetro, manteniendo los que se han mencionado como básicos. De forma que la red neuronal óptima recibe como parámetros la posición de los vértices, la velocidad y el sdf. Aunque esto mejora el aprendizaje del modelo, en el proximo apartado se valorará si el coste adicional de utilizar la velocidad en Unity lo compensa.

- A la hora de escalar el proyecto, aumentando tanto la geometría como los tipos de movimiento, resulta interesante introducir un modelo que se valga de un PCA para reducir la dimensión de los parámetros de entrada. Utilizando

el método del umbral de varianza, se calcula el mínimo de características necesarias para alcanzar un gran porcentaje de la variabilidad de los datos. En el caso de la figura por ejemplo, partiendo de un dataset con 25 vértices y 13 features por cada uno, que resultaría en 325 elementos de input, con un umbral de varianza del 95 % consigue reducirse a 45.

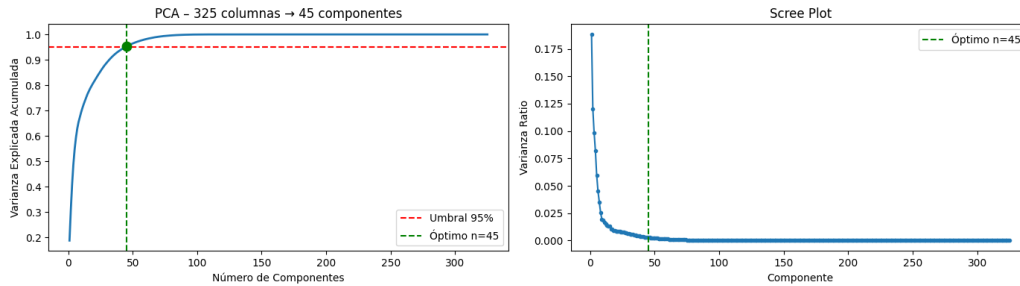


Figura 3.11: Reducción de características

3.3.5. Gestión del error acumulado

En este punto del desarrollo, los resultados de accuracy del modelo son adecuados, se alcanzan entrenamientos con un error medio por vertice de hasta 0.001m, lo que en un contexto estático sería imperceptible. Sin embargo, nuestro modelo ha de mantenerse consistente a lo largo del tiempo. A 60 fotogramas por segundo, un error tan bajo puede convertirse en una geometría completamente dañada al cabo de pocos segundos. Ante este problema, se plantean varias opciones para que el aprendizaje pueda ser capaz de autocorregirse.

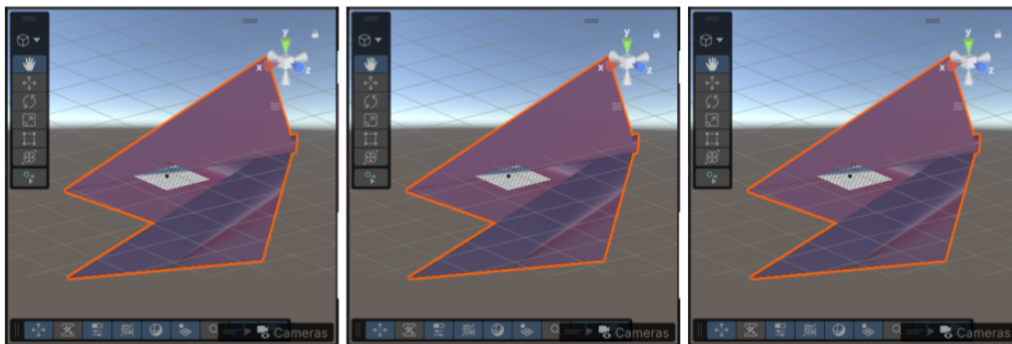


Figura 3.12: Reducción de características

- Ruido en el input:** Una de las técnicas más sencillas para mejorar la capacidad del modelo de corregir sus errores es entrenarlo creando esos pequeños errores artificialmente. Esto se consigue añadiéndole ruido a los datos de entrada, en este caso se añade un ruido gaussiano de entre un 2 y un 5 % de magnitud respecto a los datos de entrada antes de normalizar. De esta manera, el modelo aprende a corregir estos errores para corregir los suyos propios en un futuro.

- **Anchored training:** Otra forma de intentar reducir el error consiste en inferir el desplazamiento de los vértices respecto a las posiciones originales de la malla en lugar de las posiciones en el último frame ([Bertiche et al. \(2021b\)](#), [Santesteban et al. \(2022\)](#), [Holden et al. \(2019\)](#)). Este es un planteamiento intuitivo que se presenta en distintos papers mencionados previamente: DeePSD, SNUG o Subspace Neural Physics. Al pensar en simulaciones de telas, ya sea una bandera con su mástil o una prenda con su personaje, todas cuentan con unos puntos, es decir, algo a lo que anclarse. Actuando sobre esos puntos se puede asegurar que la simulación se mantendrá dentro de un espacio concreto. En esta investigación aplicamos esta técnica guardando las posiciones de los vértices en su estado inicial y aplicando los nuevos desplazamientos sobre estos. Al coger un punto de referencia del que medir los desplazamientos y no basarse en posiciones generadas, el accuracy del modelo, calculado con la diferencia entre las posiciones reales y predichas, en las pruebas de python mejora notablemente. No obstante, no se consigue alcanzar una simulación en Unity tan fiel como las producidas por la siguiente técnica.
- **Unrolling for training:** La idea de dividir el entrenamiento en distintos modelos es habitual; ya sea con una red que dé resultados más generales y una segunda que consiga refinar los detalles (arrugas, intersecciones con el colisionador, etc.) como en TailorNet [Patel et al. \(2020\)](#), o con modelos que actúen como predictor-corrector siendo el segundo normalmente en post-procesado. Sin embargo, en este proyecto se propone incluir esta idea en el propio entrenando del modelo, con la función de pérdida. El caso de [Chen et al. \(2025\)](#), por ejemplo, el concepto de una función de pérdida *force-impulse* que penaliza tanto el error en la fuerza predicha como el impulso que esta genera para las siguientes inferencias. De manera similar, en este proyecto se genera una secuencia de predicciones; la cadena de deltas generados sucesivamente permiten al modelo tener una noción sobre el cambio de velocidad y aceleración provocados.

La implementación que se incorpora en este proyecto se inspira más en la investigación de [Holden et al. \(2019\)](#), en la que se lleva a cabo un proceso similar. Además de retroalimentar los frames generados a la secuencia de input, se propaga el error utilizando la técnica BPTT (*Backpropagation Through Time*). Esta técnica consiste en retropropagar el error a lo largo de la secuencia temporal y permite a la red aprender dependencias temporales.

- **Ventana inicial:** $V_0 = [y_{t-n+1}, \dots, y_t]$
- **Bucle de predicción:** Para $i = 1, 2, \dots, W$:
 1. Calcular predicción: $\hat{y}_{t+i} = f(V_{i-1})$
 2. Actualizar ventana: $V_i = [V_{i-1} \setminus \{y_{primero}\}] \cup \{\hat{y}_{t+i}\}$
- **Función de pérdida (Loss):**

$$\mathcal{L} = \frac{1}{W} \sum_{i=1}^W (y_{t+i} - \hat{y}_{t+i})^2$$

3.3.6. Resultado e hiperparámetros

Tras el proceso de iteraciones descrito, se concluye que la red neuronal más óptima es una RNN con *unfolding*, utilizando como función de pérdida la suma entre $L1Loss$ y el la pérdida de velocidad. El input se construye con el SDF, la posición y velocidad de los vértices normalizados con inyección de ruido y como output se obtienen los deltas de desplazamiento normalizados. Por simplicidad el modelo que se ha decidido representar por pseudocódigo recoge únicamente la posición y sdf como parámetro de entrada.

Algorithm 2 Función de *Unrolling*

```

1: function RUNROLLOUT(batch_seq, batch_future)
2:    $B, W, V, F \leftarrow$  dimensiones de batch_future
3:   window  $\leftarrow$  copia(batch_seq)  $\triangleright$  La copia permite no alterar el tensor original
4:   Inyección de ruido inicial
5:    $\text{noise} \leftarrow \mathcal{N}(0, \sigma_{\text{noise}})$  del mismo tamaño que window[ $:-N :$ ]
6:   window[ $:-N :$ ]  $\leftarrow$  window[ $:-N :$ ] + noise
7:   Iteración sobre la secuencia
8:   total_loss  $\leftarrow 0$ 
9:   prev_pred_pos  $\leftarrow$  window[ $:-1, :, 0 : 3$ ]  $\triangleright$  Posición en  $t - 1$ 
10:   for  $k \leftarrow 0$  to  $W - 1$  do
11:     window_norm  $\leftarrow$  NormalizarEntrada(window)
12:     pred_delta_norm  $\leftarrow$  Modelo(window_norm)  $\triangleright$  Inferencia de la red
13:     last_pos  $\leftarrow$  window[ $:-1, :, 0 : 3$ ]
14:     pred_delta_real  $\leftarrow$  DesnormalizarDelta(pred_delta_norm)
15:     pred_pos  $\leftarrow$  last_pos + pred_delta_real  $\triangleright$  Nueva posición absoluta
16:     real_pos  $\leftarrow$  batch_future[ $:, k, :, 0 : 3$ ]
17:     real_prev_pos  $\leftarrow (k > 0) ? \text{batch\_future}[:, k - 1, :, 0 : 3] : \text{batch\_seq}[:, -1, :, 0 : 3]$ 
18:     real_delta_norm  $\leftarrow$  NormalizarDelta(real_pos - last_pos)
19:     loss_pos  $\leftarrow \mathcal{L}_{L1}(\text{pred\_delta\_norm}, \text{real\_delta\_norm})$   $\triangleright$  Cálculo del Loss de Posición
20:     pred_vel  $\leftarrow$  pred_pos - prev_pred_pos
21:     real_vel  $\leftarrow$  real_pos - real_prev_pos
22:     loss_vel  $\leftarrow \mathcal{L}_{L1}(\text{pred\_vel}/\sigma_{\text{target}}, \text{real\_vel}/\sigma_{\text{target}})$   $\triangleright$  Cálculo del Loss de Velocidad
23:     total_loss  $\leftarrow$  total_loss + loss_pos + ( $\gamma_{\text{vel}} \times \text{loss\_vel}$ )
24:     new_frame  $\leftarrow$  ConstruirFrame(pred_pos, resto de propiedades reales)
25:     window  $\leftarrow$  [window[ $:, 1 :$ ], new_frame]  $\triangleright$  Desplazar ventana temporal
26:     prev_pred_pos  $\leftarrow$  pred_pos
27:   return total_loss /  $W$ 
28: end function

```

Algorithm 3 Bucle de Entrenamiento y Validación

```

    for epoch  $\leftarrow$  1 to EPOCHS do_
1: Fase de Entrenamiento (Train)
2: Establecer modelo en modo entrenamiento
3: train_loss  $\leftarrow$  0,0 for cada (batch_seq, batch_future) en train_loader do
4:     loss  $\leftarrow$  RUNROLLOUT(batch_seq, batch_future)
5: Optimizador.zero_grad()
6: loss.backward()
7: RecortarGradientes(model.parameters, umbral = 1,0)
8: Optimizador.step()
9: train_loss  $\leftarrow$  train_loss + loss.item()
10:
11: Planificador.step() ▷ Actualizar tasa de aprendizaje
12: Fase de Evaluación (Test)
13: Establecer modelo en modo evaluación y desactivar gradientes
14: test_loss  $\leftarrow$  0,0, total_err  $\leftarrow$  0,0, total_verts  $\leftarrow$  0 for cada
    (batch_seq, batch_future) en test_loader do
15:     loss  $\leftarrow$  RUNROLLOUT(batch_seq, batch_future)
16: test_loss  $\leftarrow$  test_loss + loss.item()
▷ Métricas de error en el primer paso (k = 0)
17: pred_pos  $\leftarrow$  PredecirSiguientePaso(batch_seq)
18: target_pos  $\leftarrow$  batch_future[:, 0, :, 0 : 3]
19: distancias  $\leftarrow$  ||pred_pos - target_pos||2
20: total_err  $\leftarrow$  total_err +  $\sum$ (distancias)
21: total_verts  $\leftarrow$  total_verts + NúmeroDeElementos(distancias)
22:
23: Almacenar métricas medias del epoch en el histórico
24: Imprimir en consola: epoch, train_loss, test_loss, ErrorVértice
25:

```

Para acabar de ajustar el modelo se itera sobre los mismos datasets modificando los parámetros variables.

- **Windows size:** Cantidad de frames de la sequencia de input, esta actuará como ventana deslizando conforme se añadan los frames predichos.
- **Rollout steps:** Cantidad de frames que el modelo produce en el unroll.
- **Noise:** Se varía en la cantidad proporcional de ruido en el input normalizado (entre un 2 y un 5 %) y este se aplica a los últimos *n* frames de la ventana del input.
- **Learning rate:** Se reduce la tasa de aprendizaje cuando este se estanca.
- **Tamaño dataset:** Se varía entre datasets desde 5000 a 25000 frames para determinar el tamaño mínimo con el que se consigue un resultado apropiado.

- **Batch size:** Este depende del tamaño del dataset y el detalle que se esté dispuesto a sacrificar para reducir el tiempo de entrenamiento.

3.4. SimulacionUnity

Como se ha explicado en el último apartado, el entrenamiento genera dos archivos indispensables para la ejecución del modelo en Unity. El archivo ONNX que contiene el modelo entrenado, y el archivo JSON con los datos de normalización de los inputs y outputs del modelo. Estos archivos serán recogidos por los componentes de tipo ClothML que se han generado para remplazar el componente Cloth del objeto-tela en Unity. En este apartado se explica este proceso y sus elementos con detalle.

3.4.1. Alteraciones en la escena

Al inicio del capítulo se describe este pipeline como circular, pues se utiliza la misma escena de la simulación en Unity para recoger datos y probar posteriormente el funcionamiento del modelo entrenado. Esto requiere de ciertos cambios de configuración descritos a continuación.

- Se marcan los vértices del *Mesh Filter* como dinámicos, ya que por defecto Unity guarda las mallas en buffers estáticos. Este cambio activa una flag interna de Unity para obligar a la GPU a utilizar una estrategia de guardado más óptima para el acceso en cada frame.
- Se reemplaza el componente de simulación Cloth por el componente creado ClothML que recreará el movimiento a través del modelo cargado.
- Como se ha descrito en el estado de la cuestión, las geometrías simuladas por Cloth utilizan el *Skinned Mesh Renderer* para su renderizado. Al reemplazarlo la malla no puede ser renderizada con esta técnica, por lo que se añade un componente *Mesh Renderer* general.

3.4.2. Componente ClothML

En este apartado se profundiza en el componente mencionado, que se añade en el anexo de esta memoria.

3.4.2.1. Funcionamiento de sentis

Sentis permite importar modelos de redes neuronales entrenados a Unity y ejecutarlos en tiempo real tanto en CPU como en GPU. Para utilizar Sentis se crea un motor o *worker* que se encarga de planear las operaciones del tensor de forma paralela. Además procesa cada capa del modelo de manera secuencial, obteniendo con los datos aportados la salida del modelo.

3.4.2.2. Estructura del componente

Habiendo descrito el funcionamiento interno de la herramienta, en este apartado se explica el uso que se le ha dado para adaptarla a las necesidades del proyecto. Los componentes ClothML han sido generados a partir del *Mono Behaviour* de Unity, esta clase engloba los comportamientos de un *Game Object*. Proporciona métodos como *Start()* o *FixedUpdate()* que representan el ciclo de vida del objeto.

Start() se ejecuta una sola vez al inicio del programa, por lo que se utiliza para crear el worker e inicializar los parámetros necesarios. Los parámetros que el componente recibe son los siguientes:

- **JSON:** Contiene los parámetros de normalización. Estos se utilizarán para normalizar los datos de entrada del tensor y desnormalizar los deltas resultantes.
- **ONNX:** El modelo entrenado en formato ONNX se utiliza para inicializar el modelo de Sentis.
- **Procesador:** Se elige si el modelo va a ser ejecutado en CPU o GPU. El worker de Sentis será creado con un *BackendType* del tipo seleccionado. Se utiliza esta opción para hacer medidas de rendimiento.
- **MeshFilter:** Se accede a la malla a través de este componente.
- **Collisionador/es:** Los colliders que representan el objeto que la tela debe tener en cuenta para su colisión. Al igual que en el componente de Cloth de Unity, puede estar formado por una o más esferas y cápsulas.

Asimismo, se determinan los constraints de los puntos. Se establece el parámetro *max distance* para la lista de vértices. También se llena el history buffer con el estado de la tela quieta en su posición inicial.

FixedUpdate() es el método que Unity utiliza por defecto para la ejecución de las físicas. En él iteramos sobre los puntos de la geometría para inferir las nuevas posiciones. En la siguiente figura se visualiza una iteración para un modelo recurrente. Se accede a la malla para calcular los parámetros del frame de input, como la posición local o el SDF. Estos parámetros son normalizados con la media y la desviación típicas recibidas en el JSON. Se introducen al historial de estados, que previamente habrá liberado su primera posición, manteniendo así un tamaño constante. Se crea el tensor de las dimensiones necesarias (1 x secuencia de frames del historial x cantidad de vértices x cantidad de características) y se vuelca el contenido del historial de estados. Se ejecuta el modelo y se desnormalizan los resultados, también con los parámetros del JSON. Estos resultados finalmente se suman a la posición de los vértices en el estado del input, estableciendo así la nueva posición.

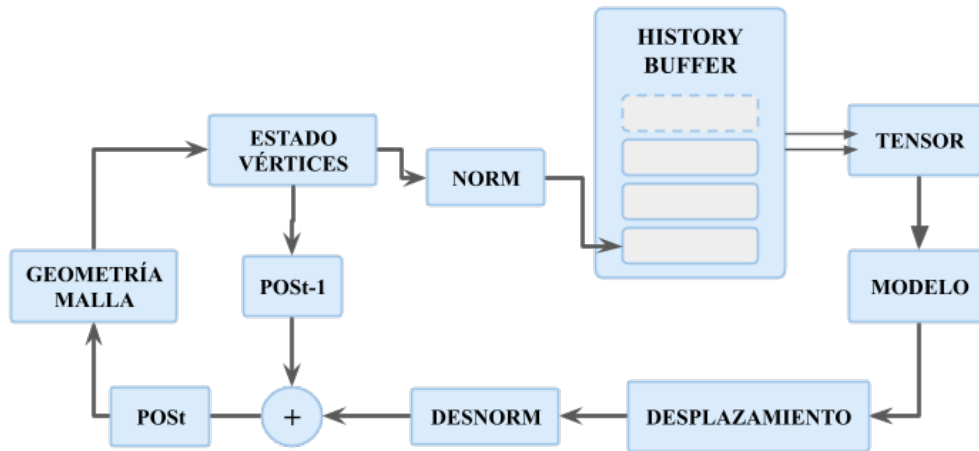


Figura 3.13: Esquema del contenido de FixedUpdate() de ClothML.

Al igual que en el caso de los notebooks del entrenamiento, se trabajó de forma paralela y existen distintas versiones de este componente en función de el modelo entrenado y las características de la tela.

- Se adapta la generación del input para ajustarse a las características que el modelo necesita.
- Los modelos MLP no requieren de historial de eventos, por lo que los datos de estado se normalizan y se transforman directamente en el tensor de input para el modelo.
- Del mismo modo, los modelos PCA que se implementaron requieren de un procesamiento específico para generar los componentes con los que se entrenó el modelo. En este caso, se aplanan el vector de entrada para estandarizarlo y proyectar los datos en el espacio de componentes que se generó al entrenar el dataset, como se puede observar en el siguiente pseudocódigo:

Algorithm 4 Procesamiento de los datos de entrada para modelos PCA

Entrada: Vértices actuales $\mathbf{V}$, Vértices anteriores $\mathbf{V}_{last}$, Parámetros de normalización θ , Matriz de componentes $\mathbf{W}$ **Salida** : Vector proyectado

```

▷ Paso 1: Construcción del vector crudo  $\mathbf{X} \in \mathbb{R}^{78}$ 
for  $v \leftarrow 0$  to  $\mathbf{V}.length$  do
  |  $\mathbf{pos} \leftarrow \mathbf{V}_v$   $\mathbf{vel} \leftarrow (\mathbf{pos} - \mathbf{V}_{last,v})/\Delta t$   $\mathbf{sdf} \leftarrow \|\mathbf{pos} - \mathbf{pos}_{ball}\| - r_{ball}$   $\mathbf{n}, \mathbf{uv} \leftarrow$ 
  | obtener de la malla Llenar bloque  $\mathbf{X}_v$  con  $\{\mathbf{pos}, \mathbf{vel}, \mathbf{sdf}, \mathbf{n}, \mathbf{maxDist}, \mathbf{uv}\}$ 

▷ Paso 2: StandardScaler
for  $f \leftarrow 0$  to 77 do
  |  $\mathbf{scaled}[f] \leftarrow \frac{\mathbf{raw}[f] - \mu_{input,f}}{\sigma_{input,f}}$ 

▷ Paso 3: Centrado PCA
for  $f \leftarrow 0$  to 77 do
  |  $\mathbf{centered}[f] \leftarrow \mathbf{scaled}[f] - \mu_{pca,f}$ 

▷ Paso 4: Proyección Lineal
for  $c \leftarrow 0$  to  $optimalN - 1$  do
  |  $dot \leftarrow 0$ ;
for  $f \leftarrow 0$  to 77 do
  |  $dot \leftarrow dot + \mathbf{W}_{c,f} \cdot \mathbf{centered}[f]$   $\mathbf{pca}[c] \leftarrow dot$ ;
return  $\mathbf{pca}$ ;
```

- El script se adapta automáticamente al resto de variaciones como número de frames de input o cantidad de vértices.

3.4.3. Resultados visuales de la inferencia

En este apartado se muestran ejemplos comparando la tela simulada por Unity y la inferencia que el modelo consigue.

AÑADIR PRUEBAS

3.4.4. Post-procesamiento

En las investigaciones más visualmente convincentes del estado del arte, se consigue llegar a ese nivel de credibilidad en la simulación gracias a estrategias de post-procesamiento que refinan la apariencia de la geometría tras ser inferida. Entre ellas destacan la corrección de intercolisiones o intersecciones con los objetos colisionadores, la reconstrucción de las normales de los vértices, la conservación de la integridad física de la tela mediante medidas de separación entre vértices o el suavizado de temblores. Al tener en este proyecto como objetivos comparar resultados de la inferencia y observar la capacidad de los modelos de replicar estas simulaciones por sí mismos, se ha querido afinar la predicción de la simulación de la manera más precisa posible fuera del motor de videojuegos. Es decir, se apreciaba medir visualmente el resultado directo del modelo sin correcciones adicionales. Esto supone resultados menos pulidos pero más significativos respecto a la capacidad de los

modelos.

La única técnica de post-procesado aplicada en este estudio es la conservación de los constraints de la tela, es decir, se tienen en cuenta los valores de *max distance* para fijar los vértices anclados y no aplicar sobre ellos ningún desplazamiento.

Conclusiones y Trabajo Futuro

4.1. Conclusiones

El objetivo de esta investigación era conseguir replicar el movimiento de una tela en un motor de videojuegos a tiempo real con baja latencia, haciéndolo accesible para desarrolladores indies y pudiéndose ejecutarse en equipos de rendimiento menor como portátiles de potencia reducida o móviles.

Se crearon simulaciones de telas con geometrías simples y movimientos dinámicos provocados por un objeto colisionador en Unity. Durante la ejecución se extrajeron los datos de interés de la tela y se convirtieron a formato CSV. A partir de estos datasets se configuró un modelo que fuera capaz de aprender el comportamiento de la tela, adoptando poses de reposo y deformaciones realizadas por el colisionador sin colapsar. En el comienzo de la investigación se plantearon seis características (que conformarían 13 features) sobre los vértices, que podían ser relevantes para capturar la esencia de la tela en la simulación: la posición, el SDF (distancia a superficie colisionadora más cercana), la velocidad, el gradiente de SDF o normales respecto al colisionador, las coordenadas UVs y la máxima distancia de movimiento desde su origen en la malla de cada vértice. Finalmente, se resaltan las siguientes características como la clave para alcanzar el resultado:

- **Reducción de features:** De las características mencionadas, las tres más útiles para recrear el movimiento fueron la posición, SDF y velocidad. Los modelos finales cuentan únicamente con ellas como input.
- **Recurrencia:** El modelo óptimo resultó ser una RNN, red neuronal recurrente. Conocer el contexto histórico de las simulaciones es esencial para dotar al modelo de cierta percepción temporal y aumentar la coherencia de la simulación.
- **Unrolling:** A esta RNN se le aplicó la técnica de “unrolling”, que permite aplicar el algoritmo de retropropagación a través del tiempo (BPTT). De esta manera, el modelo infiere una secuencia de predicciones, corrigiendo su error de manera más acertada.

- **Noise:** Modificar los datos de entrenamiento aplicandoles ruido, ha sido clave para conseguir un modelo más robusto, que pueda recuperarse de pequeños errores en la inferencia sin colapsar por el error acumulado.
- **Grandes datasets:** En la medida de lo posible, los entrenamientos con grandes cantidades de datos han demostrado naturalmente preparar al modelo para una mayor variedad de escenarios.

El modelo entrenado se exportó a formato ONNX y se cargó de nuevo en Unity, a través del paquete Sents. El componente ClothML creado consiguió sustituir el componente de Cloth de Unity, replicando en la geometría un movimiento fiel al de una tela convencional a través del modelo cargado. No sólo se puede afirmar que el modelo alcanza resultados satisfactorios a través de su “vertex error” o distancia entre posiciones predichas y reales en el testing, sino que se confirma el resultado obtenido a efecto visual: se alcanza una simulación natural y fiable al comportamiento de la tela original.

Con respecto a la reducción de latencia y tasa de frames alcanzada, se consigue llegar a doblar la tasa de frames en un ordenador de X características. ClothML, el componente generado en esta investigación permite conducir la simulación a Xfps, mientras que la tela original la limita a quedarse en Xfps.

4.1.1. Limitaciones

El estudio resultante queda con las siguientes limitaciones:

- **Dataset pequeño** en comparación con el estado del arte. Para el nivel de simplicidad de la tela empleado en la simulación, el dataset generado ha sido suficiente para generar un comportamiento fiable a nivel visual. Sin embargo se podrían producir resultados mejores, así como menos dependencia con el objeto colisionador, introduciendo más diversidad de movimiento y número de frames totales. Por limitaciones de tiempo, presupuesto y hardware, en esta investigación se ha usado una media de 5000 frames por entrenamiento, mientras que en otras investigaciones que emplean bancos de simulaciones para entrenar sus modelos llegan a usar hasta 10^6 frames [Holden et al. \(2019\)](#).
- **Topología y generalización.** El modelo puede generar buenos resultados cuando la tela empleada en el entreamiento y simulación final coinciden. No es capaz de generalizar a otras topologías o mallas dinámicas.
- **Complejidad.** Como se menciona en el apartado del Dataset, debido a la dificultad para generarlas en Unity, el modelo no ha sido probado con telas más complejas o pegadas al cuerpo como camisetas o globos con forma. Podría no resultar óptimo e incluso no producir resultados aceptables en la inferencia.
- **Optimización.** Con objeto de producir los mejores resultados posibles en el modelo y en el aspecto visual, se ha enfocado la fuerza de trabajo en el

entrenamiento y no se ha llevado a cabo una optimización exhaustiva de la aplicación del modelo en Unity. Detalles como podría ser reducir el cálculo de distancias a los colisionadores no han entrado dentro del plazo de desarrollo disponible.

Todas estas limitaciones conducen a las siguientes propuestas de trabajo futuro.

4.2. Trabajo Futuro

El trabajo a futuro de este estudio se puede centrar en varios esfuerzos:

- Realizar **pruebas de usuario** para comprobar si los resultados obtenidos se perciben como naturales. La obtención de feedback por parte de usuarios de videojuegos podría confirmar qué aspectos de la tela se han conseguido replicar y dónde se deberían producir mejoras.
- **Extender los datasets** con más situaciones y probar otros **modelos más complejos**. Han quedado por probar modelos de interés como las CNNs, GNNs y redes encoder-decoder.
- Probar con **otros tipos de tela**, otros materiales, prendas pegadas al cuerpo, y potencialmente aplicar **técnicas híbridas** de animación, simulación e inteligencia artificial para conseguir mejores resultados. Un ejemplo sería combinar técnicas de skinning con ML, atando la tela a huesos y dejando la inercia y detalles como arrugas a la inteligencia artificial.
- Añadir la posibilidad de **extraer datasets de software especializado** como Maya o Blender, e incluso de bancos externos de interés. La obtención de mejores referencias habilitaría la prueba de geometrías más complejas y de mayor calidad.

Contribuciones Personales

Eva

Muchas cosas.

Pau

Muchas cosas.

Mik

Muchas cosas.

Bibliografía

The sciences, each straining in its own direction, have hitherto harmed us little; but some day the piecing together of dissociated knowledge will open up such terrifying vistas of reality, and of our frightful position therein, that we shall either go mad from the revelation or flee from the deadly light into the peace and safety of a new dark age.

H.P. Lovecraft, *The Call of Cthulhu*

ASOBO STUDIO. Microsoft flight simulator 2024. PC / Xbox Series X/S. Xbox Game Studios, 2024.

AVALANCHE SOFTWARE. Hogwarts legacy. PC / PlayStation 5 / Xbox Series X/S. Warner Bros. Games, 2023.

BEAMNG. Beamng.drive. PC. BeamNG, 2015.

BERTICHE, H., MADADI, M. y ESCALERA, S. Pbns: Physically based neural simulation for unsupervised garment pose space deformation. *ACM Transactions on Graphics (TOG)*, vol. 40(6), páginas 1–14, 2021a.

BERTICHE, H., MADADI, M. y ESCALERA, S. Neural cloth simulation. *ACM Transactions on Graphics (TOG)*, vol. 41(6), páginas 1–14, 2022.

BERTICHE, H., MADADI, M., TYLSON, E. y ESCALERA, S. DeePSD: Automatic Deep Skinning And Pose Space Deformation For 3D Garment Animation. En *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, páginas 5451–5460. IEEE, Montreal, QC, Canada, 2021b. ISBN 978-1-6654-2812-5.

CHAUDHARY, M., POKHARIYA, C. y NARAIN, R. Towards generalized position-based dynamics. *arXiv preprint arXiv:2511.23131*, 2025.

CHEN, G., SURI, S., WU, Y., VOULGA, E., LEVIN, D. I. y PAI, D. Learning simulatable models of cloth with spatially-varying constitutive properties. *arXiv e-prints*, páginas arXiv–2507, 2025.

- CHENTANEZ, N., MACKLIN, M., MÜLLER, M., JESCHKE, S. y KIM, T. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network. *Computer Graphics Forum*, vol. 39(8), páginas 123–134, 2020. ISSN 0167-7055, 1467-8659.
- GRIGOREV, A., THOMASZEWSKI, B., BLACK, M. J. y HILLIGES, O. HOOD: Hierarchical Graphs for Generalized Modelling of Clothing Dynamics. 2023. ArXiv:2212.07242 [cs].
- HECKER, C. Physics in computer games (title only). *Communications of the ACM*, vol. 43(7), páginas 34–39, 2000.
- HOLDEN, D., DUONG, B. C., DATTA, S. y NOWROUZEZAHRAI, D. Subspace neural physics: fast data-driven interactive simulation. En *Proceedings of the 18th annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, páginas 1–12. ACM, Los Angeles California, 2019. ISBN 978-1-4503-6677-9.
- JIM HUGUNIN, U. Unite 2016 - gpu accelerated high resolution cloth simulation. <https://www.youtube.com/watch?v=kCGHX1LR3l8>, 2016. Accessed: (15/04/2026).
- KENNY ERLEBEN, K. H., JON SPORRING. *Physics-based Animation*. Charles River Media, 2005.
- KUNOS SIMULAZIONI. Assetto corsa. PC / PlayStation 4 / Xbox One. 505 Games, 2014.
- MACKLIN, M., MÜLLER, M. y CHENTANEZ, N. XPBD: position-based simulation of compliant constrained dynamics. En *Proceedings of the 9th International Conference on Motion in Games*, páginas 49–54. ACM, Burlingame California, 2016. ISBN 978-1-4503-4592-7.
- MAHMOOD, N., GHORBANI, N., TROJE, N. F., PONS-MOLL, G. y BLACK, M. J. Amass: Archive of motion capture as surface shapes. páginas 5442–5451, 2019.
- MILLINGTON, I. *Game Physics Engine Development*. CRC Press, 2007. ISBN 9781482267327.
- MÜLLER, M., HEIDELBERGER, B., HENNIX, M. y RATCLIFF, J. Position based dynamics. *Journal of Visual Communication and Image Representation*, vol. 18(2), páginas 109–118, 2007. ISSN 10473203.
- PATEL, C., LIAO, Z. y PONS-MOLL, G. Tailornet: Predicting clothing in 3d as a function of human pose, shape and garment style. En *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, páginas 7365–7375. 2020.
- PLAYGROUND GAMES. Forza horizon 6. PC / Xbox Series X/S. Xbox Game Studios, 2026.

- PROVOT, X. Deformation Constraints in a Mass Spring Model to Describe Rigid Cloth Behavior. 1995.
- PROVOT, X. ET AL. Deformation constraints in a mass-spring model to describe rigid cloth behaviour. En *Graphics interface*, páginas 147–147. Canadian Information Processing Society, 1995.
- ROCKSTAR STUDIOS. Red dead redemption 2. PlayStation 4 / Xbox One / PC. Rockstar Games, 2018.
- SANTESTEBAN, I., OTADUY, M. A. y CASAS, D. Snug: Self-supervised neural dynamic garments. En *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, páginas 8140–8150. 2022.
- UBISOFT MONTREAL. Assassin's creed unity. PC / PlayStation 4 / Xbox One. Ubisoft, 2014.
- VA, H., CHOI, M.-H. y HONG, M. Real-time cloth simulation using compute shader in unity3d for ar/vr contents. *Applied Sciences*, vol. 11(17), 2021. ISSN 2076-3417.

Licencia de uso del documento

Esta obra está bajo una licencia [Creative Commons](http://creativecommons.org/licenses/by-sa/4.0/legalcode) «Atribución-NoComercial-CompartirIgual 4.0 Internacional».



<http://creativecommons.org/licenses/by-sa/4.0/legalcode>.

Licencia de uso del código fuente

Los archivos de código fuente para generar este documento se encuentran en <https://github.com/FratosVR/Memoria-TFG> bajo la licencia [GPL-3.0](#)